Full-Stack Developer Study Roadmap

The Complete Edition · Java + Spring Boot + HTML/CSS/JS/TS + React + PostgreSQL + MongoDB + Deployment & Cloud · built to take roughly a year, one topic at a time

How this document is organized: Backend and Frontend are deliberately split into their own Parts (B and C) so you can tell, at a glance, which skills live on the server and which live in the browser — even though a full-stack developer eventually does both, learning them as two distinct tracks makes each one easier to master on its own.

PART 0 — Foundations	Java syntax · Terminal/CMD · Git & GitHub (incl. command reference) · UML · Dev Tools
PART A — Backend Engineering	OOP · Core Java Toolkit (incl. 2D/3D arrays) · Spring Boot & APIs · Databases
PART B — Frontend Engineering	HTML/CSS/JS/TS Foundations · React · Axios · API types · Frontend tooling
PART C — Deployment, Cloud & Integrations	External APIs · Docker · CI/CD · Cloud compute & cloud storage · Hosting & domains · Monitoring
PART D — Data Structures & Algorithms	Arrays · Trees · Graphs · Sorting · Dynamic Programming
PART E — Career & Interview Readiness	System design · Security · Resume/portfolio · Interview prep

What's new in the Complete Edition

- Backend and Frontend are now separate Parts (A and B) instead of interleaved phases
- Every topic lists its own subtopics, 5 hands-on projects with step-by-step instructions, and 3+ real documentation sources
- New: UML diagramming, 2D/3D Arrays, Terminal & Command Line Basics, a full Git Commands reference, and a Dev Tools & Apps roundup
- Deployment is no longer a small subsection — Part C is a full track covering Docker, CI/CD, cloud compute, and cloud storage (S3, Cloudinary, Google Cloud Storage)
- Every phase still folds into one running project — ShopFlow — so the whole year compounds into one deployed, defensible portfolio piece

Pace suggestion: this is intentionally a year-long plan. Don't rush it — two to three solid topics a week, with real running code for each, will carry you through the whole roadmap in roughly 10-12 months with room to breathe.

PART 0

Foundations

Everything to set up before Phase 1 begins

PART 0 · FOUNDATIONS

Java Anatomy & Syntax

The building blocks · read before you write your first real program

Before OOP, make sure the raw syntax is second nature. This section exists so a line like `public static void main(String[] args)` stops looking like a magic incantation and starts looking like five separate, understandable decisions.

Anatomy of a Java Program

```
public class HelloWorld { public static void main(String[] args) {
System.out.println("Hello"); } }
```

public	Access modifier — this class/method is visible from anywhere, including other packages
class HelloWorld	Declares a class named HelloWorld; the filename must match this name exactly (HelloWorld.java)
static	Belongs to the class itself, not to any instance — the JVM can call main() without creating an object first
void	This method returns no value
main	The exact method name the JVM looks for as the program's entry point
(String[] args)	An array of command-line arguments passed in when the program is run; empty if none are given
System.out.println(...)	Calls println on the standard output stream to print a line of text

Subtopics covered: *Keywords & access modifiers · naming conventions · variables & primitive types · operators · control flow · methods · compiling, running & packages*

Keywords & Access Modifiers

- Access modifiers: public, private, protected, and default (package-private, no keyword written)
- static vs instance: static belongs to the class; instance members need an object created with `new`
- final: locks a variable's value, a method against overriding, or a class against subclassing
- class / interface / extends / implements: the vocabulary for building type hierarchies (Phase 1)

Naming Conventions

- Classes & interfaces: PascalCase — Car, PaymentProcessor, Comparable

- Methods & variables: camelCase — getBalance(), totalPrice, isActive
- Constants (static final): UPPER_SNAKE_CASE — MAX_USERS, DEFAULT_TIMEOUT
- Packages: all lowercase, reverse-domain style — com.shopflow.service

Variables & Primitive Types

- Primitives: int, long, double, float, boolean, char, byte, short — stored by value
- Reference types: String, arrays, and any object — stored as a reference to memory
- Type declaration syntax: type name = value; e.g. int age = 20; double price = 19.99;
- var (local type inference, Java 10+): lets the compiler infer the type from the right-hand side

Operators

- Arithmetic: + - * / % ; integer division vs floating-point division
- Relational: == != > < >= <= ; note == compares references for objects, not content (use .equals())
- Logical: && || ! and short-circuit evaluation
- Assignment shorthand: += -= *= /= ++ --

Control Flow

- if / else if / else, and the ternary operator (condition ? a : b)
- switch statements and modern switch expressions (Java 14+, arrow syntax)
- Loops: for, enhanced for-each, while, do-while — when to reach for each
- break / continue, and labeled loops for nested breaks

Methods

- Declaration syntax: modifier returnType methodName(paramType paramName) { ... }
- Method overloading: same name, different parameter lists
- Passing by value: primitives are copied; object references are copied (but point to the same object)

Compiling, Running & Packages

- JDK vs JRE vs JVM: JDK to write/compile code, JRE to run it, JVM as the engine that executes bytecode
- javac HelloWorld.java compiles to HelloWorld.class; java HelloWorld runs it
- package declaration at the top of a file, and import to use classes from other packages
- Comments: //, /* */, and /** */ Javadoc comments for documenting public APIs

1	**Hello World Variants** — print every primitive type, then combine them into one sentence **Steps:** declare one variable per primitive type → print each with a label → combine with String.format() → run with java and confirm output
2	**Syntax Playground** — one program exercising every control-flow construct **Steps:** write an if/else chain → add a switch expression → add a for, a while, and a do-while loop → add one labeled break in a nested loop
3	**Overload Practice** — three overloaded versions of an add() method **Steps:** write add(int,int) → write add(double,double) → write add(String,String) for concatenation → call all three from main() and print results
4	**Command-Line Calculator** — read two numbers and an operator from args[] **Steps:** parse args[0] and args[2] with Integer.parseInt → read the operator from args[1] → switch on the operator → print the result, handle divide-by-zero
5	**Package Practice** — split one program across two custom packages **Steps:** create com.practice.model and com.practice.app packages → put a class in each → import one from the other → compile both with javac and run

> **■ Study Resources**
>
> • Oracle: The Java Tutorials — https://docs.oracle.com/javase/tutorial/
> • Oracle: Java Language Basics — https://docs.oracle.com/javase/tutorial/java/nutsandbolts/index.html
> • dev.java (official Java developer site) — https://dev.java/learn/

Before moving on, you should be able to: read any short Java method and explain what every keyword does; declare variables of each primitive type; write an if/else, a switch, and all three loop types from memory; and explain the difference between == and .equals() for objects.

PART 0 · FOUNDATIONS

Terminal & Command Line Basics

Navigating without a mouse · running your own programs · Windows vs Unix

Every tool from here on — javac, git, npm, docker, mvn — is run from a terminal. If typing commands still feels unfamiliar, this is the section to slow down on; everything downstream assumes it's second nature.

Subtopics covered: *navigation · file operations · running programs · environment variables & PATH · Windows CMD/PowerShell vs Unix shell*

Navigation

- pwd (Unix) / cd with no args (Windows) — print the current directory
- cd foldername to move in, cd .. to move up, cd ~ (Unix) to jump home
- ls (Unix) / dir (Windows) to list contents; ls -la for hidden files and details

File & Folder Operations

- mkdir to create a folder, touch file.txt (Unix) / type nul > file.txt (Windows) to create an empty file
- cp (Unix) / copy (Windows) to copy; mv (Unix) / move (Windows) to move or rename
- rm (Unix) / del (Windows) to delete a file; rm -r for a folder — there's no undo, be careful

Running Programs From the Terminal

- javac File.java to compile, java File to run a compiled Java class
- node file.js to run a JavaScript file directly with Node.js
- Running a project's start script: npm run dev, mvn spring-boot:run

Environment Variables & PATH

- What PATH is: the list of folders the shell searches for a command's executable
- echo $PATH (Unix) / echo %PATH% (Windows) to view it; why 'command not found' usually means a PATH issue
- Setting a variable for one session vs permanently (shell profile files vs System Environment Variables on Windows)

Windows CMD/PowerShell vs Unix Shell (bash/zsh)

- Most commands differ (dir vs ls, del vs rm) — PowerShell narrows this gap with Unix-like aliases
- Git Bash on Windows gives you a Unix-style shell without switching OS
- Tab-completion and command history (up arrow) work in all of them — use them constantly

1	**Folder Structure Builder** — recreate a small project folder tree using only the terminal **Steps:** mkdir a project folder → cd into it → mkdir src, test, and docs subfolders → touch a README.md inside it → ls -la to confirm the structure
2	**Compile-and-Run Drill** — compile and run a Java file without an IDE **Steps:** write a one-file HelloTerminal.java → cd to its folder → javac HelloTerminal.java → java HelloTerminal → confirm the .class file appeared with ls
3	**PATH Investigation** — find out where a command actually lives **Steps:** run which java (Unix) or where java (Windows) → print PATH and locate that folder in the list → temporarily remove it from PATH in one session → confirm java stops being found → restore PATH

4	**Batch File Cleanup** — use terminal commands to clean a messy folder **Steps:** create 10 dummy files with mixed extensions → list only .txt files with a wildcard → move all .txt files into a new subfolder → delete the leftovers → verify with ls
5	**Shell Profile Customization** — add a permanent shortcut command (alias) **Steps:** find your shell profile file (.zshrc/.bashrc or PowerShell profile) → add an alias like ll for ls -la → reload the profile → confirm the alias works in a new terminal tab

■ Study Resources

- MDN: Command line crash course — https://developer.mozilla.org/en-US/docs/Learn_web_development/Getting_started/Environment_setup/Command_line
- Microsoft: Windows Commands / CMD reference — https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/windows-commands
- Microsoft: PowerShell documentation — https://learn.microsoft.com/en-us/powershell/

PART 0 · FOUNDATIONS

Git & GitHub Essentials

Version control · collaboration · the workflow every dev job runs on

You'll commit your very first Phase 1 mini-project to Git, so this comes before OOP, not after. Each topic below explains what it is, why it matters, and how it actually shows up in a company — not just the command syntax.

Subtopics covered: *commit lifecycle · branching · merge vs rebase · conflict resolution · .gitignore & secrets · undoing changes · commit conventions · pull requests & review · issues & boards · branching strategy · branch protection · authentication*

Git Basics & the Commit Lifecycle

Git is a distributed version control system: it records every change you make to your code as a snapshot (a commit), and because every developer has the full history on their own machine, nobody depends on one central server just to see what happened last month. The core loop — git init/clone, edit files, git add to stage changes, git commit to save a snapshot, git log/diff to inspect history — is the single most-used skill in professional software, full stop.

> **Real-world example:** when an engineer's laptop is lost or their hard drive fails mid-sprint, they lose nothing — they clone the company's repository onto a new machine and the entire project history, down to every commit, is right there.

Branching & Branch Naming Conventions

A branch is an isolated line of development — you can build a feature, break things, and experiment freely without touching the working code everyone else depends on. Teams adopt naming conventions like feature/, bugfix/, hotfix/ prefixes so that anyone glancing at the repo instantly understands what a branch is for.

> **Real-world example:** a checkout platform might have a branch named feature/paymongo-webhook-retry sitting alongside hotfix/cart-total-rounding-bug — the naming alone tells the whole team which one is routine and which one needs eyes today.

Merging vs Rebasing

Both combine work from one branch into another, but they tell a different story afterward. A merge keeps the full, real history. A rebase rewrites your branch's commits so they look like they were built directly on top of the latest main, producing a clean, linear history but losing the record of exactly when the branch diverged.

> **Real-world example:** many companies configure GitHub's 'squash and merge' button as the only allowed merge strategy, so every feature — no matter how many messy work-in-progress commits it had — lands on main as one clean commit.

Resolving Merge Conflicts

A conflict happens when two branches change the same lines of the same file and Git can't automatically decide which version is correct, so it stops and asks you to choose. This is completely normal on any team of more than one person.

> **Real-world example:** two engineers both edit the same shipping-fee calculation in the same week; when the second person's PR is merged, Git flags the conflicting lines, and the two quickly hop on a call to agree on the final logic.

.gitignore & Keeping Secrets Out of Git

A .gitignore file tells Git which files to never track — build output, dependency folders, IDE settings, and critically, files holding passwords or API keys. Once a secret is committed, it's in the project's history forever unless the history itself is rewritten.

> **Real-world example:** a startup once nearly leaked a live database password because a .env file was committed by accident; afterward, every new repo was required to start from a template with a strict .gitignore plus automated secret-scanning.

Undoing Changes: Stash, Reset & Revert

git stash temporarily shelves uncommitted changes. git reset rewrites history — powerful, but dangerous the moment anyone else has already pulled that history. git revert instead creates a brand-new commit that undoes an earlier one, leaving the original history intact and auditable.

> **Real-world example:** when a bad deploy reaches production, the team runs git revert on the merge commit instead of force-pushing a reset, so the fix is visible in history and nobody else's work gets silently broken.

Commit Message Conventions (Conventional Commits)

A structured format like feat: add coupon validation or fix: correct tax rounding error makes history scannable at a glance, and tooling can read that structure to auto-generate changelogs or decide version bumps.

> **Real-world example:** the Angular team's commit convention is well known precisely because their release notes and semantic version numbers are generated automatically from commit messages.

Pull Requests & Code Review

A pull request (PR) is a formal proposal to merge one branch into another: before code ever reaches main, at least one other engineer reviews it, asks questions, and can request changes. This is where bugs get caught before customers ever see them.

> **Real-world example:** almost every tech company requires at least one approving review before a PR can merge — a new hire's first real lesson in the codebase's conventions often comes from review comments on their first PR.

GitHub Issues & Project Boards

Issues track individual bugs, features, or tasks with discussion attached directly to the code they concern; Project boards arrange issues into Kanban-style columns. Linking a PR to an issue lets merging it automatically close the issue.

> **Real-world example:** a small startup runs its whole two-week sprint planning through a GitHub Project board — product, design, and engineering all look at the same board instead of a separate tracker that drifts out of sync.

Branching Strategy: Git Flow vs GitHub Flow / Trunk-Based

Git Flow uses long-lived develop, release, and hotfix branches and suits scheduled, versioned releases. GitHub Flow / trunk-based keeps a single main branch always deployable, with short-lived feature branches merged frequently — the norm at fast-moving SaaS companies.

> **Real-world example:** a company shipping a public SaaS product might deploy from main dozens of times a day behind feature flags, while a bank's core-banking team still uses formal Git Flow release branches for compliance.

Branch Protection Rules & Required CI Checks

GitHub lets a repo owner lock the main branch so nobody — regardless of seniority — can push directly to it; every change must go through a PR, pass required checks, and get required approvals.

> **Real-world example:** a company configures main so that even its own CTO cannot push directly to it — every change must pass CI and get an approving review first, preventing one bad Friday-afternoon push from breaking the site.

Authentication: SSH Keys & Personal Access Tokens

SSH keys let you authenticate to GitHub without typing a password on every push. Personal Access Tokens (PATs) serve scripts and CI systems, and can be scoped to specific permissions and revoked individually.

> **Real-world example:** a CI pipeline authenticates to pull a private repository using a scoped Personal Access Token rather than any engineer's personal login, so it can be revoked instantly without locking out a real person's account.

1	**Repo From Day One** — start using Git before Phase 1 even begins **Steps:** git init in your first mini-project folder → commit after every working increment → write a one-line README → push it to a new GitHub repo
2	**Solo Feature Branch + PR** — practice the branch-then-PR habit even working alone **Steps:** create a feature/ branch for one concept project → commit your work there → open a PR against your own main → read your own diff like a reviewer would, then merge
3	**Conventional Commit Practice** — build the feat/fix/chore habit early **Steps:** make 5 small changes to any project → commit each with a feat:, fix:, refactor:, or chore: prefix → run git log --oneline to review the result
4	**Merge Conflict Simulation** — deliberately create and resolve a conflict **Steps:** edit the same file/line on two branches → try merging one into the other → read the conflict markers → resolve manually and commit the resolution
5	**Branch-Protected Practice Repo** — configure GitHub's protection rules yourself **Steps:** create a new GitHub repo → turn on branch protection for main → require a PR and one passing check before merge → confirm a direct push to main is now blocked

■ Study Resources

- Pro Git Book (free, official) — https://git-scm.com/book/en/v2
- Git official documentation — https://git-scm.com/doc
- GitHub Docs — https://docs.github.com/en

PART 0 · FOUNDATIONS · CHEAT SHEET

Git Commands Reference

The commands behind every topic above, grouped by what you're trying to do

git init	Turn the current folder into a new Git repository
git clone	Copy a remote repository (and its full history) to your machine
git status	Show what's staged, unstaged, and untracked right now
git add / git add .	Stage a specific file, or stage every changed file
git commit -m "message"	Save a snapshot of staged changes with a message
git log / git log --oneline	View commit history, full or condensed to one line each
git diff	Show unstaged changes line by line
git branch	List local branches; git branch creates a new one
git checkout -b / git switch -c	Create and switch to a new branch
git merge	Merge the named branch into your current branch
git rebase	Replay your commits on top of the named branch
git stash / git stash pop	Shelve uncommitted changes, then restore them later
git reset --soft/--mixed/--hard	Move the branch pointer back; soft keeps changes staged, hard discards them
git revert	Create a new commit that undoes a previous one, safely
git push / git push -u origin	Upload commits to the remote; -u sets the default upstream
git pull	Fetch and merge the remote branch into your current one
git fetch	Download remote changes without merging them yet
git remote -v	List the remote repositories this local repo is connected to
git tag	Mark a specific commit (e.g. a release) with a permanent label
git cherry-pick	Apply one specific commit from another branch onto yours
git blame	Show who last changed each line of a file, and in which commit
git log --grep="text"	Search commit messages for a keyword

Pin this page. Every command here maps directly back to a topic in the Git & GitHub Essentials section above — if a command feels unfamiliar, that's the topic to re-read.

PART 0 · FOUNDATIONS

UML Basics

Drawing your design before you code it · the shared language of software structure

UML (Unified Modeling Language) is a standardized set of diagram types for visualizing how a system is structured and how it behaves, before or alongside writing the code. You won't use every UML diagram type day to day, but class diagrams and sequence diagrams in particular come up constantly — in design docs, in whiteboard interviews, and in onboarding new teammates to a codebase.

Subtopics covered: *class diagrams · use case diagrams · sequence diagrams · activity diagrams · ER diagrams*

Class Diagrams

- Boxes represent classes: name, attributes, and methods in three stacked sections
- Relationship lines: association, aggregation (hollow diamond), composition (filled diamond), inheritance (hollow triangle arrow)
- Multiplicity notation (1, 0..1, *, 1..*) describing how many of one class relate to another

Use Case Diagrams

- Actors (stick figures) representing users or external systems
- Ovals representing actions the system supports ("Place Order", "Cancel Subscription")
- Good for scoping a feature with a non-technical stakeholder before any code exists

Sequence Diagrams

- Vertical lifelines representing objects/services, time flowing top to bottom
- Horizontal arrows representing method calls or messages between them
- Extremely useful for mapping out an API request across Controller → Service → Repository → DB

Activity Diagrams

- Flowchart-style diagram for business logic: start node, decision diamonds, end node
- Good for modeling a multi-step process like checkout or order fulfillment

ER Diagrams (Entity-Relationship)

- Not strictly UML, but the same visual thinking applied to database schemas
- Entities as tables, attributes as columns, lines showing foreign key relationships and cardinality
- The natural bridge into Part A's Databases topic — draw this before writing a single CREATE TABLE

1	**Class Diagram: ShopFlow Product Hierarchy** — diagram Phase 1's Product/Cart classes before writing them **Steps:** list every class you plan to build → add attributes and key methods to each box → draw inheritance arrows for Product subclasses → draw composition for Cart/CartItem → then implement the code and compare
2	**Use Case Diagram: Checkout Flow** — scope the checkout feature from the user's perspective **Steps:** draw a Customer actor → add ovals for Browse, Add to Cart, Apply Coupon, Checkout → connect each to the actor → add a Payment Provider as a second actor for the payment use case

3	**Sequence Diagram: Login Request** — trace a JWT login call across every layer **Steps:** draw lifelines for Client, Controller, Service, Repository, DB → add the login request arrow from Client → add each internal call in order → add the JWT response arrow back to Client
4	**Activity Diagram: Order Fulfillment** — model the PENDING to SHIPPED order lifecycle **Steps:** add a start node → add decision diamonds for payment success/failure → add activity boxes for each state transition → add an end node for both success and failure paths
5	**ER Diagram: ShopFlow Schema** — diagram the database before Part A's Databases topic **Steps:** list entities: users, products, categories, orders, order_items → add key attributes to each → draw foreign key lines between related entities → mark one-to-many vs many-to-many relationships

> **■ Study Resources**
>
> • UML Diagrams reference (uml-diagrams.org) — https://www.uml-diagrams.org/
> • OMG: Official UML Specification — https://www.omg.org/spec/UML/
> • PlantUML documentation (text-based UML) — https://plantuml.com/

You already track your roadmap on Trello and sketch planning diagrams in Mermaid AI — Mermaid's class and sequence diagram syntax is a fast, text-based way to practice everything in this section without switching tools.

PART 0 · FOUNDATIONS · REFERENCE

Dev Tools & Apps Roundup

The apps that sit around your code, not in it

You don't need every tool below on day one — this is a reference to come back to as each need shows up naturally through the roadmap.

IDE / Editor	IntelliJ IDEA (Java/Spring Boot), VS Code (frontend, general-purpose)
API Testing	Postman, Insomnia — manually test endpoints before wiring up the frontend
Database Client	DBeaver or pgAdmin (PostgreSQL), MongoDB Compass (MongoDB)
Diagramming	Mermaid (text-based, fast), draw.io / diagrams.net, Lucidchart
Project Tracking	Trello, GitHub Projects, Notion
Code Quality	SonarLint (in-IDE), ESLint + Prettier (JS/TS)
Design / Mockups	Figma — useful once Part B's frontend work starts needing a visual target
Terminal Enhancement	Git Bash (Windows), Oh My Zsh (Unix) — nicer prompts and shortcuts
API Documentation	Swagger/OpenAPI (auto-generated from your Spring Boot endpoints)

Your current setup — IntelliJ IDEA, Trello for roadmap tracking, Mermaid AI for planning diagrams, and SonarLint for code quality — already covers the IDE, Project Tracking, Diagramming, and Code Quality rows above. The rest of this table fills in as Part A (Databases), Part B (Figma for frontend mockups), and Part C (API testing, documentation) come up.

PART A

Backend Engineering

Everything that runs on the server: Java, Spring Boot, and your databases

PHASE 1 OF 10

OOP Fundamentals

Encapsulation · Inheritance · Abstraction · Polymorphism · Composition · Exceptions · SOLID · Design Patterns

The goal of this phase is active recall through code, not passive reading. For each pillar below, build several of the suggested projects in IntelliJ before moving on — the point is to force yourself to write the pattern from memory, not to build a complete app.

Encapsulation

Hiding internal state behind a controlled public interface, so a class's data can only change through methods that validate it.

Subtopics: *private fields · public getters/setters · validation inside setters · immutability basics*

1	**Bank Account** — private balance, public deposit/withdraw with validation **Steps:** make balance private → add deposit(amount) rejecting negatives → add withdraw(amount) rejecting overdrafts → add getBalance() → test edge cases in main()
2	**Student Grade Book** — private grades list exposed only through safe methods **Steps:** make a private List grades → add addGrade() validating 0-100 → add getAverage() and getHighest() → never expose the raw list → test with 5 sample grades
3	**Temperature Sensor** — hide the raw reading, expose converted units only **Steps:** store a private raw Celsius reading → add getTemperatureCelsius() → add getTemperatureFahrenheit() doing the math internally → add a private validate() for out-of-range values
4	**Immutable Point Class** — practice full immutability, not just private fields **Steps:** make x and y private final → set both only via constructor → provide no setters → add a translate() that returns a NEW Point instead of mutating
5	**Password Vault** — encapsulate sensitive data with a masked accessor **Steps:** store a private password field → add setPassword() with a minimum-length check → add getMaskedPassword() returning **** instead of the real value → add a package-private getRealPassword() for internal use only

■ Study Resources

- Oracle: Classes and Objects (Encapsulation) — https://docs.oracle.com/javase/tutorial/java/javaOO/classvars.html
- GeeksforGeeks: Encapsulation in Java — https://www.geeksforgeeks.org/java/encapsulation-in-java/
- Baeldung: Encapsulation in Java — https://www.baeldung.com/java-encapsulation

Inheritance

Letting one class reuse and extend another's fields and behavior, modeling an is-a relationship.

Subtopics: *extends keyword · super() calls · method overriding · protected access*

1	**Animal Hierarchy** — Animal base class with Dog, Cat, Bird subclasses **Steps:** write an abstract-ish Animal with name and speak() → extend with Dog, Cat, Bird → override speak() and move() per subclass → call super() in each constructor → loop over an Animal[] printing each speak()
2	**Employee Payroll** — Employee base with FullTime, PartTime, Contractor **Steps:** write Employee with name and calculatePay() → extend with 3 subclasses → override calculatePay() differently per type → write an HR class that only knows about Employee
3	**Vehicle Registry** — Vehicle base with Car, Truck, Motorcycle **Steps:** give Vehicle shared fields plate and year → extend with 3 subclasses → override fuelCost() per type → print a registry report iterating a Vehicle list
4	**Shape Hierarchy with Protected Fields** — practice protected access across subclasses **Steps:** make Shape with a protected color field → extend with Circle and Square → let subclasses read (not the outside world) color directly → add a describe() using the protected field
5	**Multi-Level Inheritance Chain** — practice a 3-generation hierarchy **Steps:** write LivingThing → extend with Animal → extend Animal with Dog → call super() correctly at every level → confirm Dog inherits from both ancestors

■ Study Resources

- Oracle: Inheritance (Java Tutorials) — https://docs.oracle.com/javase/tutorial/java/IandI/subclasses.html
- GeeksforGeeks: Inheritance in Java — https://www.geeksforgeeks.org/java/inheritance-in-java/
- Baeldung: Inheritance in Java — https://www.baeldung.com/java-inheritance

Abstraction

Exposing only the essential behavior through an abstract class or interface, hiding implementation detail the caller doesn't need.

Subtopics: *abstract classes · abstract methods · interfaces · programming to an interface*

1	**Payment Gateway** — abstract Payment with CreditCard, GCash, PayMaya **Steps:** write abstract class Payment with abstract process() → implement 3 subclasses → write a Checkout class using only the Payment type → swap implementations without changing Checkout
2	**Notification System** — Notifier interface with Email, SMS, Push **Steps:** define interface Notifier with send(String msg) → implement EmailNotifier, SMSNotifier, PushNotifier → write client code that only depends on Notifier → loop through a List sending one message
3	**Shape Area Calculator** — abstract Shape with polymorphic getArea() **Steps:** write abstract class Shape with abstract getArea() → implement Circle, Rectangle, Triangle → store them in a List → sum all areas with one loop, no type-checking

4	**Database Connector Interface** — abstract away which database you're actually using **Steps:** define interface DataStore with save()/find() → implement an InMemoryStore → implement a stub FileStore → write a service that only knows about DataStore
5	**Media Player Abstraction** — one play() call, many formats **Steps:** define interface MediaPlayer with play(String file) → implement Mp3Player and Mp4Player → write a Playlist class using only MediaPlayer → add a third format without touching Playlist

■ Study Resources

- Oracle: Interfaces and Abstract Classes — https://docs.oracle.com/javase/tutorial/java/IandI/abstract.html
- GeeksforGeeks: Abstraction in Java — https://www.geeksforgeeks.org/java/abstraction-in-java-2/
- Baeldung: Abstract Classes in Java — https://www.baeldung.com/java-abstract-class

Polymorphism

Calling the same method name on different object types and getting behavior specific to each — the payoff of inheritance and abstraction working together.

Subtopics: *method overriding · dynamic dispatch · upcasting · the @Override annotation*

1	**Discount Engine** — one apply(price) call, three discount types **Steps:** define DiscountStrategy interface with apply(double price) → implement Regular/Senior/StudentDiscount → store them in a List → loop and print each discounted price
2	**Logger Switcher** — swap logging destinations without changing call sites **Steps:** define Logger interface with log(String msg) → implement Console/File/DatabaseLogger → write one service class taking a Logger → swap implementations at runtime via constructor injection
3	**Game Character Moves** — Warrior, Mage, Archer each override attack() **Steps:** write abstract Character with abstract attack() → extend with 3 subclasses → write a game loop calling attack() on any Character → confirm each prints different combat text
4	**Shape Renderer** — polymorphic draw() across shape types **Steps:** reuse or extend the Shape hierarchy from Abstraction → add draw() to each subclass → iterate a List calling draw() once → confirm no if/instanceof checks were needed
5	**Employee Bonus Calculator** — override one method, change behavior per subclass **Steps:** reuse the Employee hierarchy from Inheritance → add calculateBonus() to the base class → override it per subclass with different logic → sum total payroll cost with one loop over Employee

■ Study Resources

- Oracle: Polymorphism (Java Tutorials) — https://docs.oracle.com/javase/tutorial/java/IandI/polymorphism.html
- GeeksforGeeks: Polymorphism in Java — https://www.geeksforgeeks.org/java/polymorphism-in-java/
- Baeldung: Polymorphism in Java — https://www.baeldung.com/java-polymorphism

Composition

Building complex objects out of smaller ones (has-a) instead of inheritance (is-a) — often the more flexible choice.

Subtopics: *has-a relationships · composing objects · delegation · composition vs inheritance*

1	**Computer Builder** — Computer has-a CPU, RAM, Storage, GPU **Steps:** write small classes for CPU, RAM, Storage, GPU → compose them as fields inside Computer → build 2 different configs by swapping components → print a spec sheet by delegating to each part
2	**Restaurant Order** — Order has-a list of OrderItem, each has-a MenuItem **Steps:** write MenuItem with name and price → write OrderItem composing a MenuItem plus quantity → write Order composing a List → compute total by delegating down the object graph
3	**User Profile** — User has-a Address, ContactInfo, Preferences **Steps:** write Address, ContactInfo, Preferences as separate classes → compose all three into User → update just the Address without touching the rest of User → print a full profile by delegating to each part
4	**Car Engine Swap** — prove composition's flexibility over inheritance **Steps:** write an Engine interface with start() → compose an Engine field inside Car → implement GasEngine and ElectricEngine → swap engines on the same Car instance at runtime
5	**Pizza Builder** — compose a pizza from independent toppings **Steps:** write a Topping class → compose a List inside Pizza → add/remove toppings after construction → compute price by summing each Topping's cost

■ Study Resources

• GeeksforGeeks: Composition in Java — https://www.geeksforgeeks.org/java/composition-in-java/
• Baeldung: Composition, Aggregation, Association — https://www.baeldung.com/java-composition-aggregation-association
• Refactoring.Guru: Inheritance vs Composition — https://refactoring.guru/design-patterns/composition-over-inheritance

Exceptions

Signaling and handling error conditions with a structured, typed alternative to error codes or silent failure.

Subtopics: *try/catch/finally · custom exception classes · throwing vs catching · exception hierarchies*

1	**ATM Simulator** — custom exceptions for real banking error cases **Steps:** define InsufficientFundsException, InvalidPINException, DailyLimitExceededException → throw each from the right validation point → catch and handle each distinctly → add a finally block logging every attempt
2	**File Parser** — a custom exception carrying extra context **Steps:** read a CSV line by line → define MalformedRowException with a row-number field → throw it on a bad row → let the caller decide to skip or abort on catch
3	**User Registration** — an exception hierarchy for validation **Steps:** define a base ValidationException → extend with EmptyFieldException, DuplicateEmailException, WeakPasswordException → throw the specific one per failure → catch the base type once at the top level
4	**Retry Wrapper** — recover from a transient failure automatically **Steps:** write a method that fails randomly ~30% of the time → wrap calls to it in a try/catch loop → retry up to 3 times before giving up → log each retry attempt

5	**Resource Cleanup Demo** — guarantee cleanup even when something throws **Steps:** open a fake "resource" (e.g. a Scanner on a file) → force an exception mid-processing → use try-with-resources to guarantee it closes → confirm cleanup ran even on the failure path

> **■ Study Resources**
>
> • Oracle: Exceptions (Java Tutorials) — https://docs.oracle.com/javase/tutorial/essential/exceptions/index.html
> • Baeldung: Exception Handling in Java — https://www.baeldung.com/java-exceptions
> • GeeksforGeeks: Custom Exceptions in Java — https://www.geeksforgeeks.org/java/user-defined-custom-exception-in-java/

SOLID Principles & Design Patterns

After the pillar projects above, apply SOLID principles across everything you write. For design patterns, implement at least one pattern per category below.

Subtopics: *Single Responsibility · Open/Closed · Liskov Substitution · Interface Segregation · Dependency Inversion · Creational patterns · Structural patterns · Behavioral patterns*

Creational	Factory Method, Builder, Singleton	Control how objects are created
Structural	Decorator, Adapter, Composite	Compose objects into larger structures
Behavioral	Observer, Strategy, Command, Template Method	Define how objects interact and behave

1	**Factory Method: ProductFactory** — centralize object creation logic **Steps:** define a Product interface → write 2-3 concrete Product types → write ProductFactory.create(type) returning the right one → confirm callers never use `new` directly
2	**Builder: CartBuilder** — construct a complex object step by step **Steps:** identify an object with many optional fields (a Cart) → write a Builder with chainable with...() methods → add a build() returning the finished Cart → compare readability vs a giant constructor
3	**Observer: StockObserver** — notify dependents automatically on state change **Steps:** define a Subject interface with subscribe()/notify() → implement it on a Product's stock field → write a StockObserver that prints an alert → trigger it by lowering stock below a threshold
4	**Strategy: DiscountStrategy Swap** — change an algorithm at runtime without if/else **Steps:** reuse the DiscountStrategy interface from Polymorphism → inject a different strategy into the same Cart instance → confirm behavior changes with zero Cart code changes
5	**Decorator: Coffee Order Add-ons** — add behavior without subclass explosion **Steps:** write a base Coffee class with cost() → write a decorator abstract class wrapping a Coffee → implement MilkDecorator, SyrupDecorator → stack decorators and confirm cost() adds up correctly

■ Study Resources

- Oracle: Java OOP Concepts — https://docs.oracle.com/javase/tutorial/java/concepts/index.html
- Refactoring.Guru: Design Patterns — https://refactoring.guru/design-patterns
- Baeldung: SOLID Principles in Java — https://www.baeldung.com/solid-principles

Final project for this phase

ShopFlow — Foundation (Mini E-Commerce Core)

- Product hierarchy: PhysicalProduct, DigitalProduct (inheritance + polymorphism)
- CartItem composes Product + quantity + discount (composition)
- DiscountStrategy interface: PercentOff, FixedOff, NoDiscount (Strategy pattern)
- ProductFactory.create(type) to instantiate correct subclass (Factory Method)
- CartBuilder for constructing Cart with optional coupon/address (Builder)
- Private cart total, expose getTotal() only (encapsulation)
- Custom exceptions: OutOfStockException, InvalidQuantityException
- Observer: StockObserver prints alert when quantity drops below threshold

Build a console-based shopping cart system that forces you to apply everything from this phase. No Spring Boot yet — pure Java. This is your OOP proof-of-concept before adding any framework.

PHASE 2 OF 10

Core Java Toolkit

Collections · Generics · Exceptions · File I/O · Streams · JDBC · 2D/3D Arrays · Build Tools · Concurrency · Testing · Logging

This is the layer most self-taught devs skip, then feel shaky about in interviews. Everything here is what Spring Boot, JPA, and your own future code leans on constantly. Master this before Phase 3 and the framework will feel like a natural extension instead of a black box.

2D & 3D Arrays

Arrays of arrays — the foundation for grids, matrices, and board-shaped data before you ever reach for a Collections-based structure.

Subtopics: *declaring int[][]/int[][][] · nested loops for traversal · jagged arrays · matrix operations*

1	**Matrix Addition & Multiplication** — classic 2D array math **Steps:** declare two int[][] matrices → write addMatrices() summing element-wise → write multiplyMatrices() with the row-by-column algorithm → print both results in grid form
2	**Tic-Tac-Toe Board** — a 2D array as game state **Steps:** represent the board as char[3][3] → write a printBoard() method → write placeMove(row,col,player) → write checkWinner() scanning rows/cols/diagonals
3	**Image as Pixel Grid** — treat a 2D int array as grayscale pixel data **Steps:** build a small int[][] representing pixel brightness → write a method to invert all values → write a method to rotate the grid 90 degrees → print before/after grids
4	**Conway's Game of Life** — a 2D grid that evolves by neighbor-counting rules **Steps:** represent the grid as boolean[][] → write countLiveNeighbors(row,col) → write one generation-step method applying the rules → print several generations in sequence
5	**3D Grid: Rubik's Cube State** — extend into three dimensions **Steps:** represent one cube face's stickers as a 3D array (or a small voxel grid) → write a method to print one layer at a time → write a method to rotate one layer → verify state stays consistent after a rotation

■ Study Resources

- Oracle: Arrays (Java Tutorials) — https://docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html
- GeeksforGeeks: Multidimensional Arrays in Java — https://www.geeksforgeeks.org/java/multidimensional-arrays-in-java/
- Baeldung: Multi-Dimensional Arrays in Java — https://www.baeldung.com/java-multi-dimensional-arrays

Java Collections Framework

The standard library's ready-made data structures — List, Set, Map, Queue — that replace most hand-rolled array logic.

Subtopics: *List (ArrayList, LinkedList) · Set (HashSet, TreeSet) · Map (HashMap, TreeMap) · Queue/Deque · Iterator · Comparable vs Comparator*

1	**Inventory Manager** — store and look up items two ways **Steps:** store products in a List → also index them in a Map by ID → add search-by-category filtering → compare List vs Map lookup speed conceptually
2	**Word Frequency Counter** — classic HashMap counting exercise **Steps:** read a block of text → split into words → count occurrences with a HashMap → sort results by frequency using a Comparator
3	**Task Scheduler** — PriorityQueue ordering by urgency **Steps:** define a Task class with a priority field → add tasks to a PriorityQueue → implement Comparable on Task or pass a Comparator → poll tasks off in priority order
4	**Duplicate Detector** — compare Set implementations **Steps:** load a dataset with duplicates into a HashSet → load the same data into a TreeSet → compare iteration order between the two → print which duplicates were caught
5	**Custom Comparator Suite** — sort one list three different ways **Steps:** build a List → sort by price using Comparator.comparing() → sort by name alphabetically → chain a 2-level sort: category then price

■ Study Resources

- Oracle: Collections Framework Overview — https://docs.oracle.com/javase/tutorial/collections/index.html
- Oracle: Java Collections API (Javadoc) — https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Collection.html
- Baeldung: Guide to the Java ArrayList — https://www.baeldung.com/java-arraylist

Generics

Writing type-safe, reusable classes and methods that work across many types without casting.

Subtopics: *generic classes · bounded types · wildcards (? extends / ? super) · generic interfaces*

1	**Generic Box / Pair** — the simplest possible generic container **Steps:** write class Box with a get/set → write class Pair holding two related values → instantiate both with different types → confirm the compiler blocks a wrong-type insert
2	**Generic Repository Interface** — preview what Spring Data will do for you later **Steps:** define interface Repository → add save(T entity), findById(ID id), findAll() → implement it in-memory with a Map → instantiate it for a Product entity
3	**Bounded Type Sorter** — a generic method with a type constraint **Steps:** write > void sort(List list) → implement a simple sort algorithm inside it → call it with a List and a List → confirm it rejects a non-Comparable type
4	**Generic Stack Implementation** — a reusable stack for any type **Steps:** write class Stack backed by an ArrayList → add push(), pop(), peek(), isEmpty() → test it with Integers and with Strings → add bounds-checking for pop() on empty
5	**Wildcard Practice: printAll()** — understand ? extends vs ? super **Steps:** write void printAll(List list) → call it with List and List → write a second method using ? super Integer for adding → explain in a comment why each wildcard was needed

■ Study Resources

- Oracle: Generics (Java Tutorials) — https://docs.oracle.com/javase/tutorial/java/generics/index.html
- Baeldung: Java Generics — https://www.baeldung.com/java-generics
- GeeksforGeeks: Generics in Java — https://www.geeksforgeeks.org/java/generics-in-java/

Exception Handling — Deep Dive

Beyond try/catch basics: the mechanics that matter for production code and interviews.

Subtopics: *checked vs unchecked · try-with-resources · multi-catch · exception chaining*

1	**Resource-Safe File Reader** — guarantee cleanup across multiple resources **Steps:** open two files with try-with-resources in one statement → force a failure reading the second → confirm both still close via AutoCloseable → print a summary of what was read before failure
2	**Retry-on-Exception Utility** — a generic, reusable retry wrapper **Steps:** write a functional-interface-based retry(Supplier, int attempts) → catch a specific exception type inside it → retry up to N times with a short delay → throw the last exception if all attempts fail
3	**Multi-Catch Practice** — handle several exception types in one block **Steps:** write a method that can throw either IOException or NumberFormatException → catch both in a single multi-catch block → log which type actually occurred → rethrow wrapped in a custom exception

4	**Exception Chaining Logger** — preserve the original cause **Steps:** catch a low-level exception → wrap it in a custom exception via `new CustomEx(msg, cause)` → log the full chain with getCause() → confirm the original stack trace is still visible
5	**Checked vs Unchecked Refactor** — convert one to the other and feel the difference **Steps:** write a method throwing a checked exception → note every caller is now forced to handle it → refactor it to a RuntimeException subclass instead → note callers are no longer forced — discuss which is better here

■ Study Resources

- Oracle: Exceptions (Java Tutorials) — https://docs.oracle.com/javase/tutorial/essential/exceptions/index.html
- Baeldung: Checked vs Unchecked Exceptions — https://www.baeldung.com/java-checked-unchecked-exceptions
- Baeldung: try-with-resources — https://www.baeldung.com/java-try-with-resources

File I/O

Reading and writing data outside the running program — text files, CSVs, and config files.

Subtopics: *java.io vs java.nio.file · BufferedReader/Writer · reading/writing CSV · serialization basics*

1	**CSV Import/Export** — round-trip your product catalog through a file **Steps:** load product data from a CSV with BufferedReader → parse each line into a Product object → edit the in-memory list → write it back out with BufferedWriter
2	**Simple Log Writer** — append timestamped entries to a growing file **Steps:** open a file in append mode → write a timestamped line on each call → rotate to a new file after 100 lines → read the log back and print the last 5 lines
3	**Config File Loader** — read key=value pairs into a Map **Steps:** write a small .properties-style file by hand → read it line by line → split each line on the first = → load results into a Map
4	**Directory Walker** — use java.nio.file.Files to explore a folder tree **Steps:** use Files.walk() on a project directory → filter to only .java files → count total lines across all of them → print the 3 largest files by size
5	**Word Search File Tool** — search multiple files for a keyword **Steps:** accept a folder path and a keyword → read every .txt file in it → print filename + line number for each match → handle a missing folder gracefully with a caught exception

■ Study Resources

- Oracle: File I/O (java.nio.file) — https://docs.oracle.com/javase/tutorial/essential/io/fileio.html
- Oracle: Basic I/O (java.io) — https://docs.oracle.com/javase/tutorial/essential/io/index.html
- Baeldung: Java - Write to File — https://www.baeldung.com/java-write-to-file

Java Streams

A functional-style pipeline for processing collections — source, then a chain of operations, then a result.

Subtopics: *map/filter/sorted/distinct/flatMap · Collectors (toList, groupingBy, joining) · reduce · when to use a plain loop instead*

1	**Product Query Pipeline** — filter, map, and sort in one chain **Steps:** start from a List → filter() by category → map() to a lighter DTO → sorted() by price, collect() to a List
2	**Sales Report Grouping** — groupingBy + summing in one line **Steps:** start from a List → groupingBy(category) → downstream collector summingDouble(price) → print the resulting Map
3	**Word Stats Pipeline** — chain multiple Stream operations on text **Steps:** split a paragraph into words → stream().distinct().sorted() → collect(Collectors.joining(", ")) → print the unique sorted word list

4	**Refactor Sprint** — turn 3 old loops into Stream pipelines **Steps:** pick 3 loops from earlier Phase 1/2 projects → rewrite each as a Stream pipeline → compare line count and readability → note any case where the loop was actually clearer
5	**Custom Collector Practice** — go beyond the built-in collectors **Steps:** build a List → use Collectors.partitioningBy() to split in-stock vs out-of-stock → use Collectors.averagingDouble() for average price → print both results

■ Study Resources

• Oracle: java.util.stream (Javadoc) — https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/stream/Stream.html
• Baeldung: Java 8 Streams Introduction — https://www.baeldung.com/java-8-streams-introduction
• Oracle: Aggregate Operations (Streams tutorial) — https://docs.oracle.com/javase/tutorial/collections/streams/index.html

JDBC

Raw, unabstracted database connectivity — do this before Spring Data/Hibernate so you see exactly what JPA will later hide from you.

Subtopics: *DriverManager & Connection · PreparedStatement · ResultSet mapping · transactions (commit/rollback)*

1	**Raw JDBC CRUD** — hand-write every SQL operation **Steps:** connect to PostgreSQL with DriverManager → write INSERT via PreparedStatement → write SELECT and map ResultSet rows to objects → write UPDATE and DELETE
2	**SQL Injection Demo (then fix it)** — see the risk, then remove it **Steps:** build a query with raw string concatenation → demonstrate an injection payload breaking it → rewrite using PreparedStatement placeholders → confirm the same payload is now inert
3	**ResultSet to POJO Mapper** — manual object-relational mapping **Steps:** run a SELECT * FROM products → manually map each column to a Product field → return a List → compare this by hand to what JPA will automate in Phase 4
4	**Transfer Transaction** — atomic multi-statement operations **Steps:** set autoCommit(false) → run two UPDATE statements (debit one row, credit another) → commit() on success → force a failure and confirm rollback() restores both rows
5	**Connection Pool Awareness Demo** — feel why raw JDBC doesn't scale **Steps:** open and close a new Connection in a loop 50 times → time how long that takes → read about HikariCP's pooling approach → write a comment explaining what a pool would fix here

■ Study Resources

- Oracle: JDBC Basics (Java Tutorials) — https://docs.oracle.com/javase/tutorial/jdbc/basics/index.html
- Baeldung: Introduction to JDBC — https://www.baeldung.com/java-jdbc
- PostgreSQL JDBC Driver Documentation — https://jdbc.postgresql.org/documentation/

Build Tools

Maven and Gradle — you'll use one of these on every Java job, since Spring Boot itself is generated from a template built on one of them.

Subtopics: *Maven (pom.xml) · Gradle (build.gradle) · dependency management · standard project layout*

1	**Convert to Maven** — restructure a plain-Java project properly **Steps:** create a standard src/main/java, src/test/java layout → write a pom.xml with your project's coordinates → add one real dependency (e.g. JUnit) → build with mvn package and confirm a JAR is produced
2	**Dependency Conflict Investigation** — see version conflicts firsthand **Steps:** add two dependencies that share a transitive dependency at different versions → run mvn dependency:tree → identify which version actually gets used → explain why in a comment
3	**Same Project in Gradle** — compare the two ecosystems directly **Steps:** rebuild the same small project with build.gradle instead → add the same dependency → run gradle build → write 3 bullet points comparing the experience to Maven

4	**Custom Maven Profile** — environment-specific configuration **Steps:** add a dev profile and a prod profile to pom.xml → vary one property between them → build with -P dev and -P prod → confirm the right property was used each time
5	**Multi-Module Maven Project** — practice a realistic multi-module layout **Steps:** create a parent pom with two modules (core and api) → let api depend on core → build the whole project with one command from the parent → confirm both modules compile in the right order

■ Study Resources

- Apache Maven: Getting Started Guide — https://maven.apache.org/guides/getting-started/
- Gradle User Manual — https://docs.gradle.org/current/userguide/userguide.html
- Baeldung: Introduction to Maven — https://www.baeldung.com/maven

Testing (JUnit & Mockito)

Writing automated tests that prove your code works — and keep proving it after every change.

Subtopics: *JUnit 5 (@Test, assertions) · Mockito (@Mock, when/thenReturn) · Arrange-Act-Assert · unit vs integration tests*

1	**Test the Cart Logic** — unit test pure business logic **Steps:** write JUnit tests for a discount calculation method → cover the normal case → cover a zero/negative edge case → run with mvn test and confirm all pass
2	**Mock the Repository** — test a service in isolation **Steps:** write a Service depending on a Repository interface → mock the Repository with @Mock → stub a return value with when().thenReturn() → verify() the repository method was called exactly once
3	**Exception Path Testing** — prove your error handling actually works **Steps:** write a method that throws a custom exception on bad input → use assertThrows() to test it → assert on the exception's message → add a matching happy-path test alongside it
4	**Parameterized Tests** — run one test logic against many inputs **Steps:** use @ParameterizedTest with @ValueSource or @CsvSource → test a validation method against 5 different inputs → confirm both valid and invalid cases are covered → run and read the per-case output
5	**Test Coverage Pass** — measure and improve coverage on one class **Steps:** pick a class with no tests yet → write tests until coverage tooling shows 80%+ → identify the one branch that was hardest to cover → note why in a comment

■ Study Resources

- JUnit 5 User Guide — https://junit.org/junit5/docs/current/user-guide/
- Mockito official documentation — https://site.mockito.org/
- Baeldung: Mockito Series — https://www.baeldung.com/mockito-series

Concurrency & Multithreading (Basics)

Awareness-level multithreading — enough to recognize the terms and use the basic tools every backend job expects.

Subtopics: *Thread vs Runnable · ExecutorService & thread pools · race conditions · synchronized keyword*

1	**Basic Thread Demo** — the simplest possible multithreading **Steps:** implement Runnable with a small task → start 3 threads running it → print each thread's name as it runs → join() all three before the program exits
2	**Parallel File Processor** — use a thread pool instead of raw threads **Steps:** create an ExecutorService with a fixed pool → submit one task per file to process → collect results with Future → shut down the executor cleanly
3	**Race Condition Demo** — reproduce the bug on purpose **Steps:** share one int counter across 100 threads incrementing it → run without synchronization → observe the final count is wrong → explain why in a comment

4	**Fix It With Synchronized** — the simplest fix for the race condition above **Steps:** add the synchronized keyword to the increment method → rerun the same 100-thread test → confirm the final count is now correct every time → note the performance trade-off
5	**Producer-Consumer with a Queue** — a classic concurrency pattern **Steps:** use a thread-safe queue (e.g. LinkedBlockingQueue) → write a producer thread adding items → write a consumer thread removing and processing them → run both concurrently and confirm no items are lost

■ Study Resources

- Oracle: Concurrency (Java Tutorials) — https://docs.oracle.com/javase/tutorial/essential/concurrency/index.html
- Baeldung: Java ExecutorService Guide — https://www.baeldung.com/java-executor-service-tutorial
- GeeksforGeeks: Multithreading in Java — https://www.geeksforgeeks.org/java/multithreading-in-java/

Logging

Replacing println debugging with real, leveled, configurable logging — the way production code actually reports what's happening.

Subtopics: *log levels (TRACE/DEBUG/INFO/WARN/ERROR) · SLF4J + Logback · log patterns/formatting · what never to log*

1	**Add Logging to ShopFlow** — replace every println with real logging **Steps:** add SLF4J + Logback to your project → replace println calls with logger.info()/debug() → add a logger.error() with the exception on every catch block → confirm log output includes a timestamp and level
2	**Log Level Filtering Demo** — see levels actually filter output **Steps:** log one message at each level (TRACE through ERROR) → set the configured level to WARN → confirm only WARN and ERROR print → change the config to DEBUG and rerun
3	**Structured Log Pattern** — customize the log output format **Steps:** edit the Logback pattern to include class name and thread → rerun the app and inspect the new format → explain what each pattern token means in a comment
4	**Rolling File Appender** — log to a file that rotates automatically **Steps:** configure a RollingFileAppender in logback.xml → set a max file size trigger → generate enough log volume to trigger a rotation → confirm a second log file was created
5	**Redact-Sensitive-Data Filter** — practice never logging secrets **Steps:** write a method that logs a user object → confirm it currently would log a password field → add a toString() override or DTO that masks it → confirm logs now show **** instead

■ Study Resources

- SLF4J User Manual — https://www.slf4j.org/manual.html
- Logback documentation — https://logback.qos.ch/documentation.html
- Baeldung: Introduction to SLF4J — https://www.baeldung.com/slf4j-with-log4j2-logback

Final project for this phase

ShopFlow — Core Java Service Layer

- Rebuild the cart's data layer using Collections (Map catalog) and 2D-array-backed inventory grid
- Generic Repository implemented in-memory, ready to be swapped for JPA later
- Full custom exception hierarchy (ShopFlowException base) used consistently
- Load/save the product catalog to a CSV file with File I/O
- Rewrite catalog filtering/search/reporting logic using Java Streams
- Raw JDBC version that persists products to PostgreSQL — no ORM yet
- Restructure as a Maven build with a src/test/java suite of JUnit + Mockito tests
- Replace all console output with SLF4J logging at appropriate levels

Same ShopFlow domain, now backed by real collections, real files, real Streams, and a real (if manual) database connection. Phase 3 replaces the manual JDBC with Spring Data.

PHASE 3 OF 10

Framework & API

Spring Boot · REST · GraphQL · JWT · Layered architecture

Wire your OOP and Core Java knowledge into a real framework. Build endpoints one at a time, test each in Postman before moving to the next. Understand why the layers exist — Controller receives, Service decides, Repository stores.

Spring Core & Dependency Injection

The container that wires your objects together, so classes declare what they need instead of constructing it themselves.

Subtopics: *@Component/@Service/@Repository · @Autowired & constructor injection · the Spring container/ApplicationContext · Controller → Service → Repository layering*

1	**First Spring Boot App** — get the container running end to end **Steps:** generate a project with Spring Initializr → add one @RestController with a hello endpoint → run it and hit it in a browser → add one @Service and inject it into the controller
2	**Constructor Injection Refactor** — the recommended DI style **Steps:** take a class using field @Autowired → refactor to constructor injection instead → confirm Spring still wires it correctly → explain in a comment why constructor injection is preferred
3	**Three-Layer Wiring Practice** — Controller → Service → Repository from scratch **Steps:** write a Repository interface with an in-memory implementation → write a Service depending on the Repository → write a Controller depending on the Service → trace one request through all three layers
4	**Bean Scope Experiment** — singleton vs prototype **Steps:** create a @Component with a counter field → inject it into two different controllers → confirm both share the same instance (singleton default) → change scope to prototype and confirm they no longer share state
5	**Configuration Properties Class** — type-safe app config **Steps:** add custom values to application.properties → bind them to a @ConfigurationProperties class → inject that class where needed → confirm changing the properties file changes behavior without code changes

■ Study Resources

- Spring Boot Reference Documentation — https://docs.spring.io/spring-boot/index.html
- Spring Framework: Core Technologies (DI) — https://docs.spring.io/spring-framework/reference/core/beans.html
- Spring Guides: Getting Started — https://spring.io/guides

REST API Design

Designing HTTP endpoints that follow predictable, resource-based conventions other developers can guess without reading docs.

Subtopics: *HTTP verbs & status codes · resource naming · DTOs vs entities · pagination & filtering*

1	**Todo REST API** — the canonical CRUD starter **Steps:** build GET/POST/PUT/DELETE for a Todo resource → return correct status codes (200/201/204/404) → add a DTO separate from the entity → test every endpoint in Postman
2	**Product Catalog API** — pagination, sorting, filtering **Steps:** add GET /products with page and size query params → add sorting by a query param → add filtering by category → return a wrapped response with total count + page data
3	**Consistent Error Response** — one predictable error shape **Steps:** define a standard ErrorResponse DTO (status, message, timestamp) → return it from a 404 case → return it from a 400 validation case → confirm both look consistent in Postman
4	**Resource Naming Audit** — fix bad REST naming on purpose, then correctly **Steps:** write 5 badly-named endpoints (verbs in the URL, inconsistent plurals) → rewrite each following REST convention → compare both lists side by side → explain the reasoning for each fix
5	**HATEOAS-lite Response** — add discoverability to a response **Steps:** return a resource plus a _links section → include a self link and a related-resource link → confirm a client could navigate using only the links → keep it simple — full HATEOAS isn't required

■ Study Resources

- MDN: HTTP Overview — https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview
- Spring: Building a RESTful Web Service — https://spring.io/guides/gs/rest-service
- RESTful API Design (restfulapi.net) — https://restfulapi.net/

Request Validation & Exception Handling

Rejecting bad input before it reaches your business logic, and turning every failure into a clean, structured response.

Subtopics: *@Valid & Bean Validation annotations · custom constraint annotations · @ControllerAdvice · @ExceptionHandler*

1	**Validated Registration Endpoint** — the standard @Valid workflow **Steps:** add @NotBlank, @Email, @Size to a RegisterRequest DTO → annotate the controller param with @Valid → trigger a validation failure and inspect the default error → confirm valid input passes through
2	**Global Exception Handler** — one place for every error **Steps:** write a @ControllerAdvice class → add @ExceptionHandler for MethodArgumentNotValidException → add one for your custom ShopFlowException → add a catch-all for unexpected 500s
3	**Custom Validation Annotation** — beyond the built-in constraints **Steps:** design a rule the built-ins don't cover (e.g. @ValidSKU) → implement a ConstraintValidator for it → apply it to a DTO field → test both a passing and failing value
4	**Field-Level Error Messages** — return exactly which field failed and why **Steps:** catch MethodArgumentNotValidException in your handler → loop over its field errors → build a Map of field → message → return that map in the error response body
5	**Nested Object Validation** — validate an object inside another object **Steps:** create a DTO containing a nested Address object → add @Valid on the nested field too → trigger a failure on a nested field specifically → confirm the error correctly names the nested path

■ Study Resources

- Spring: Validation (Bean Validation) — https://docs.spring.io/spring-framework/reference/core/validation/beanvalidation.html
- Baeldung: Validation in Spring Boot — https://www.baeldung.com/spring-boot-bean-validation
- Baeldung: @ControllerAdvice and @ExceptionHandler — https://www.baeldung.com/exception-handling-for-rest-with-spring

JWT Authentication

Stateless authentication: the server issues a signed token instead of maintaining a session, and every request proves identity by presenting it.

Subtopics: *token generation & signing · the Spring Security filter chain · securing endpoints · refresh tokens*

1	**User Auth Service** — the full register/login/JWT flow **Steps:** build /register hashing the password with BCrypt → build /login verifying credentials → generate a signed JWT on successful login → build a /me endpoint requiring a valid token
2	**Custom JWT Filter** — understand the filter chain, not just the annotation **Steps:** write a filter extending OncePerRequestFilter → extract and validate the token from the Authorization header → set the SecurityContext on success → register the filter in the security config

3	**Role-Based Endpoint Protection** — not just "logged in" but "allowed" **Steps:** add a role claim to the JWT (e.g. USER/ADMIN) → protect one endpoint with @PreAuthorize for ADMIN only → confirm a USER token gets a 403 → confirm an ADMIN token succeeds
4	**Refresh Token Flow** — keep users logged in safely **Steps:** issue a short-lived access token and a longer-lived refresh token → build a /refresh endpoint accepting the refresh token → issue a new access token from it → invalidate refresh tokens on logout
5	**Token Expiry & Error Handling** — handle the expired-token case gracefully **Steps:** set a short expiry for testing (e.g. 30 seconds) → call a protected endpoint after it expires → confirm you get a clean 401, not a stack trace → add a specific error message distinguishing expired from invalid

■ Study Resources

- jwt.io: Introduction to JSON Web Tokens — https://jwt.io/introduction
- Spring Security Reference — https://docs.spring.io/spring-security/reference/index.html
- Baeldung: JWT with Spring Security — https://www.baeldung.com/spring-security-oauth-jwt

GraphQL

A query language for APIs where the client specifies exactly what shape of data it wants back, in a single request.

Subtopics: *schema-first design · queries vs mutations · resolvers/DataFetchers · comparing the feel vs REST*

1	**GraphQL Notes API** — the schema-first starter **Steps:** write a .graphqls schema for a Note type → add a Query for notes and note(id) → add a Mutation for createNote → implement each resolver/DataFetcher
2	**Nested Query Resolution** — the signature GraphQL feature **Steps:** add a related type (e.g. Author on Note) → write a resolver that fetches the nested Author only when requested → query notes with and without the author field → confirm the nested fetch only runs when asked for
3	**Product Search as GraphQL** — port a REST endpoint over **Steps:** take your REST product search endpoint → design an equivalent GraphQL query with filters as arguments → implement the resolver reusing your existing Service → compare the two response shapes side by side
4	**GraphQL Error Handling** — structured errors in the GraphQL way **Steps:** trigger a resolver exception on purpose → inspect the default GraphQL error response shape → customize it to include a clean error code → confirm partial data still returns alongside the error where valid
5	**REST vs GraphQL Comparison Doc** — write down what you actually noticed **Steps:** implement the same feature both ways (you likely already have) → note request/response size differences → note over-fetching/under-fetching differences → write 5 bullet points on when you'd pick each

■ Study Resources

- GraphQL.org: Official Learning Guide — https://graphql.org/learn/
- Spring for GraphQL Reference — https://docs.spring.io/spring-graphql/reference/index.html
- Baeldung: Guide to GraphQL with Spring Boot — https://www.baeldung.com/spring-graphql

Testing Spring Boot Applications

Testing across Spring's layers — from a pure unit test to a full application-context integration test.

Subtopics: *@SpringBootTest · MockMvc for controller tests · @DataJpaTest for repository tests · test profiles*

1	**Controller Test with MockMvc** — test an endpoint without a running server **Steps:** write a @WebMvcTest for one controller → mock the Service layer it depends on → perform a GET and assert the JSON response → perform a POST with a bad body and assert 400
2	**Full Integration Test** — boot the whole application context **Steps:** write a @SpringBootTest with a real (test) database → call an endpoint through TestRestTemplate or MockMvc → assert data was actually persisted → clean up test data after each test
3	**Repository Test with @DataJpaTest** — test just the persistence layer **Steps:** write a @DataJpaTest for one Repository → save an entity and query it back → assert a custom query method returns the right rows → confirm this test runs against an in-memory or test DB

4	**Test Profile Configuration** — separate config for tests vs real runs **Steps:** create an application-test.properties → point it at a test database → activate it with @ActiveProfiles("test") → confirm your real dev database is never touched by tests
5	**Security-Aware Controller Test** — test a JWT-protected endpoint **Steps:** write a test hitting a secured endpoint with no token → assert 401 → generate a valid test token and retry → assert 200 with the expected body

■ Study Resources

- Spring Boot: Testing Reference — https://docs.spring.io/spring-boot/reference/testing/index.html
- Baeldung: Testing in Spring Boot — https://www.baeldung.com/spring-boot-testing
- JUnit 5 User Guide — https://junit.org/junit5/docs/current/user-guide/

Final project for this phase

ShopFlow — Backend API Layer

- Product CRUD: POST /products, GET /products/{id}, PATCH, DELETE
- Cart endpoints: add item, remove item, apply coupon, get total
- Order placement: POST /orders, transitions (PENDING → CONFIRMED → SHIPPED)
- JWT auth: register, login, secure all endpoints except GET /products
- Global @ControllerAdvice returning structured JSON errors
- GraphQL alternative exposing product search
- Unit + integration tests: service layer with Mockito, MockMvc for controllers, ≥80% coverage on business logic

Extend your Phase 2 core-Java service layer into a full Spring Boot API. Keep the OOP design — now expose it over HTTP, and let Spring Data replace your manual JDBC.

PHASE 4 OF 10

Databases

PostgreSQL · MongoDB · JPA/Hibernate · N+1 · Query optimization

The phase most people rush and regret later. Hibernate behavior — lazy loading, session scope, N+1 — causes invisible production bugs. Your Phase 2 raw-JDBC work makes it obvious what Hibernate does for you here.

JPA Entity Mapping & Relationships

Turning Java classes into database tables declaratively, and modeling how those tables relate to each other.

Subtopics: *@Entity/@Id/@GeneratedValue · @Column/@Table · @OneToMany/@ManyToOne · @ManyToMany · mappedBy & @JoinColumn*

1	**Basic Entity Mapping** — your first JPA entity **Steps:** annotate a Product class with @Entity → add @Id @GeneratedValue on the primary key → add @Column for a couple of fields → save and fetch one row via a Repository
2	**One-to-Many: Category → Products** — the most common relationship **Steps:** add a Category entity → add @OneToMany on Category, @ManyToOne on Product → set mappedBy correctly on the owning side → save a category with 3 products and fetch it back
3	**Many-to-Many: Products ↔ Tags** — a join-table relationship **Steps:** add a Tag entity → add @ManyToMany on both Product and Tag → let JPA generate the join table → add and remove tags from a product and confirm the join table updates
4	**Bidirectional Relationship Pitfall** — see and fix the classic mistake **Steps:** set up a bidirectional @OneToMany/@ManyToOne without syncing both sides → trigger the inconsistency (only one side updates) → fix it with a helper method that sets both sides together → confirm both sides now stay in sync
5	**Embedded Value Object** — @Embeddable for a value that isn't its own entity **Steps:** create an Address class without its own @Id → annotate it @Embeddable → embed it inside User with @Embedded → confirm Address columns land directly on the users table

■ Study Resources

- Spring Data JPA Reference — https://docs.spring.io/spring-data/jpa/reference/
- Hibernate ORM Documentation — https://hibernate.org/orm/documentation/
- Baeldung: JPA Entity Mapping — https://www.baeldung.com/jpa-entities

Lazy vs Eager Loading & the N+1 Problem

The Hibernate behavior that causes the most invisible production bugs: how and when related data actually gets fetched.

Subtopics: *FetchType.LAZY vs EAGER · detecting N+1 via SQL logs · JOIN FETCH · @EntityGraph*

1	**Reproduce N+1 On Purpose** — see the bug before you fix it **Steps:** enable Hibernate SQL logging → fetch a list of 10 orders, each with lazy-loaded items → loop over them accessing .getItems() → count the queries fired — confirm it's 1 + N, not 1
2	**Fix With JOIN FETCH** — the JPQL-level fix **Steps:** write a custom @Query using JOIN FETCH on the association → rerun the same loop → confirm it's now a single query → compare the SQL log before and after
3	**Fix With @EntityGraph** — the annotation-based fix **Steps:** add an @EntityGraph to the repository method instead → specify the attribute to eagerly fetch → rerun and confirm the same single-query result → note the difference in approach vs JOIN FETCH
4	**LAZY vs EAGER Default Comparison** — feel the difference directly **Steps:** set a @OneToMany to FetchType.EAGER → fetch the parent without touching the children → confirm the children loaded anyway (extra query or join) → switch back to LAZY and confirm they don't load until accessed
5	**LazyInitializationException Demo** — reproduce and fix the classic Hibernate error **Steps:** fetch an entity inside a @Transactional service method → return it to a controller that accesses a lazy field after the session closed → reproduce the LazyInitializationException → fix it with JOIN FETCH or a DTO projection instead

■ Study Resources

- Baeldung: N+1 Problem in Hibernate — https://www.baeldung.com/hibernate-common-performance-problems-in-logs
- Hibernate ORM: Fetching Documentation — https://hibernate.org/orm/documentation/
- Spring Data JPA: @EntityGraph — https://docs.spring.io/spring-data/jpa/reference/jpa/entity-graph.html

JPQL & Native Queries

Writing queries against your entity model (JPQL) or dropping down to raw SQL (native) when you need to.

Subtopics: *@Query with JPQL · native SQL queries · named parameters · when to choose each*

1	**Custom JPQL Finder Methods** — beyond what method-name derivation can do **Steps:** write a @Query using JPQL for a multi-condition search → use named parameters (:category, :minPrice) → call it from a Service → compare readability vs a derived method name
2	**Native SQL for a Complex Report** — when JPQL isn't enough **Steps:** write a @Query(nativeQuery = true) for a report JPQL can't express cleanly → map the result to a projection interface or DTO → run it and confirm correct results → note why native was necessary here

3	**Derived Query Methods** — the zero-JPQL option **Steps:** write findByCategoryAndPriceLessThan() as a derived method name → confirm Spring Data generates the query automatically → compare it to writing the same thing in JPQL → list one query too complex for this approach
4	**Pagination in a Custom Query** — combine @Query with Pageable **Steps:** write a JPQL query taking a Pageable parameter → return a Page → call it with different page/size values → confirm total count and page metadata are correct
5	**Update Query** — modify data without loading entities first **Steps:** write a @Modifying @Query that bulk-updates a field → call it and confirm rows changed without fetching them first → note this bypasses the persistence context — explain the implication

■ Study Resources

- Spring Data JPA: Query Methods — https://docs.spring.io/spring-data/jpa/reference/jpa/query-methods.html
- Baeldung: JPQL vs Native Query — https://www.baeldung.com/spring-data-jpa-query
- Jakarta Persistence: JPQL Language Reference — https://jakarta.ee/specifications/persistence/

PostgreSQL Indexing & Query Optimization

Making queries fast at scale — understanding what an index actually does and how to prove a query is using one.

Subtopics: *B-tree vs GIN indexes · EXPLAIN ANALYZE · composite indexes · when an index doesn't help*

1	**Baseline Before Optimizing** — measure first, then improve **Steps:** seed a table with 100,000+ rows → run a common query and record its EXPLAIN ANALYZE output → note the execution time and whether it's doing a sequential scan
2	**Add a Single-Column Index** — the most common fix **Steps:** CREATE INDEX on the column your WHERE clause filters on → rerun EXPLAIN ANALYZE → confirm it now uses an index scan → compare execution time before/after
3	**Composite Index for a Multi-Column Query** — order matters **Steps:** create a composite index on (category, price) → run a query filtering on both columns → confirm the composite index is used → test filtering on price alone and see if it's still used
4	**GIN Index for Text Search** — beyond B-tree **Steps:** try a LIKE '%term%' search and note it can't use a normal index → add a GIN index for full-text search → rerun the search using to_tsvector/to_tsquery → compare performance to the LIKE version
5	**When an Index Doesn't Help** — the negative case matters too **Steps:** add an index on a low-cardinality column (e.g. a boolean) → run EXPLAIN ANALYZE on a query filtering it → observe Postgres may still choose a sequential scan → explain why in a comment

■ Study Resources

- PostgreSQL Official Documentation: Indexes — https://www.postgresql.org/docs/current/indexes.html
- PostgreSQL: Using EXPLAIN — https://www.postgresql.org/docs/current/using-explain.html
- Use The Index, Luke! (indexing guide) — https://use-the-index-luke.com/

Transactions

Grouping multiple database operations so they either all succeed or all fail together — no partial updates.

Subtopics: *@Transactional · propagation levels · rollback rules · isolation basics*

1	**Basic @Transactional Service Method** — the default happy path **Steps:** write a service method performing 2 related writes → annotate it @Transactional → force a failure after the first write → confirm both writes rolled back together
2	**Rollback-For Customization** — control which exceptions trigger a rollback **Steps:** throw a checked exception inside a @Transactional method → confirm by default it does NOT roll back → add rollbackFor = Exception.class → confirm it now does roll back
3	**Propagation: REQUIRES_NEW** — a nested transaction that commits independently **Steps:** call a REQUIRES_NEW method from within another @Transactional method → fail the outer transaction after the inner one runs → confirm the inner transaction's write survived anyway → explain why in a comment
4	**Read-Only Transaction** — an optimization for query-only methods **Steps:** mark a read-only service method @Transactional(readOnly = true) → confirm it still works correctly → explain what this hints to Hibernate/the DB driver
5	**Transfer Funds End-to-End** — the classic transactional example, in Spring **Steps:** write a transferFunds(fromId, toId, amount) service method → debit one account, credit another, inside one @Transactional method → force a failure mid-transfer → confirm neither account's balance changed

■ Study Resources

- Spring: Transaction Management — https://docs.spring.io/spring-framework/reference/data-access/transaction.html
- Baeldung: Spring @Transactional — https://www.baeldung.com/transaction-configuration-with-jpa-and-spring
- PostgreSQL: Transaction Isolation — https://www.postgresql.org/docs/current/transaction-iso.html

MongoDB & Document Databases

A schema-flexible alternative to relational tables — storing nested, denormalized documents instead of normalized rows.

Subtopics: *document model (embedding vs referencing) · MongoRepository · custom @Query · aggregation pipelines*

1	**First MongoDB Document** — the basics of a document-based entity **Steps:** annotate a Review class with @Document → write a MongoRepository → save and fetch a document → compare its shape to a JPA entity
2	**Embedding vs Referencing** — the core MongoDB design decision **Steps:** embed a short list of reviews directly inside a Product document → separately model a User referenced only by ID → explain in a comment when you'd choose each approach → query both and compare

3	**Custom @Query in MongoDB** — beyond derived methods **Steps:** write a MongoRepository method using @Query with a Mongo query string → filter by a nested field → confirm results match a manual check → compare to the JPQL equivalent from earlier
4	**Aggregation Pipeline: Average Rating** — MongoDB's answer to GROUP BY **Steps:** write an aggregation pipeline grouping reviews by productId → compute an average rating per product → run it via MongoTemplate → print the results sorted by rating
5	**Product + Reviews Integration** — connect this to your PostgreSQL data **Steps:** keep products in PostgreSQL as before → store their reviews in MongoDB, referenced by productId → write a service method combining both in one response → confirm the two databases stay in sync on review add

■ Study Resources

- MongoDB Official Documentation — https://www.mongodb.com/docs/
- Spring Data MongoDB Reference — https://docs.spring.io/spring-data/mongodb/reference/
- MongoDB: Aggregation Pipeline Guide — https://www.mongodb.com/docs/manual/core/aggregation-pipeline/

Schema Migrations (Flyway)

Version-controlling your database schema the same way you version-control your code, so every environment evolves in lockstep.

Subtopics: *versioned migration files · the migrate command · safe vs unsafe migrations · baseline & repair*

1	**First Flyway Migration** — the basic workflow **Steps:** add Flyway as a dependency → write V1__create_products_table.sql → run the app and confirm Flyway applied it automatically → check the flyway_schema_history table it created
2	**Add a Column Safely** — V2 without breaking V1 data **Steps:** write V2__add_stock_column.sql adding a nullable column → run the migration against existing data → confirm no existing rows broke → backfill a default value in a follow-up statement
3	**Rename Column the Safe Way** — a two-step migration to avoid downtime **Steps:** write a migration adding the new column name → write a follow-up migration copying data across → write a final migration dropping the old column → explain why this is safer than a direct rename in one step
4	**Migration Failure Recovery** — what happens when a migration breaks **Steps:** write a migration with a deliberate SQL error → run it and observe Flyway marks it failed → fix the SQL → use repair (or fix manually) and confirm the next run succeeds
5	**Baseline an Existing Database** — adopting Flyway on a pre-existing schema **Steps:** create a database schema manually (skipping Flyway) → add Flyway to the project pointing at it → run flyway baseline to mark the current state as V1 → confirm future migrations apply cleanly on top

■ Study Resources

- Redgate Flyway Documentation — https://documentation.red-gate.com/fd/
- Baeldung: Database Migrations with Flyway — https://www.baeldung.com/database-migrations-with-flyway
- Spring Boot: Database Initialization (Flyway/Liquibase) — https://docs.spring.io/spring-boot/how-to/data-initialization.html

Final project for this phase

ShopFlow — Database Layer

- Normalized PostgreSQL schema: users, products, categories, orders, order_items
- JPA entities with proper relationships and fetch strategies
- Fix N+1 on order history endpoint using @EntityGraph
- Flyway migrations for every schema change from here forward
- MongoDB collection: product_reviews with embedded rating + text, referenced userId
- Index: products.category_id, orders.user_id, orders.created_at
- EXPLAIN ANALYZE the 3 most-used queries and optimize each
- Wrap multi-step operations (checkout, stock transfer) in @Transactional with correct rollback rules

Connect Phase 3's API to a real PostgreSQL database. Add MongoDB for product reviews.

Part A complete. You now have a fully working backend: a layered Spring Boot API, secured with JWT, backed by PostgreSQL and MongoDB, tested, and built on a solid Core Java foundation. Part B switches tracks entirely to what runs in the browser.

PART B

Frontend Engineering

Everything that runs in the browser: HTML, CSS, JavaScript, TypeScript, and React

PHASE 5 OF 10

Frontend Foundations

HTML · CSS · Bootstrap/Tailwind · JavaScript (ES6+) · TypeScript

Before React, get fluent in the raw languages the browser actually runs. This is what lets you debug a layout bug or a broken event handler without guessing — and it's what separates someone who can use React from someone who understands it.

HTML

The structural skeleton of every web page — semantic, accessible markup that means something, not just divs everywhere.

Subtopics: *semantic HTML5 elements · forms & validation attributes · accessibility (ARIA, alt text) · meta tags & SEO basics*

1	**Semantic Portfolio Page** — structure before style **Steps:** use header, nav, main, section, article, footer correctly → no CSS yet — structure only → validate it with the W3C HTML validator → view the outline in browser dev tools
2	**Accessible Contact Form** — forms that work for everyone **Steps:** add proper for every input → use the right input types (email, tel, etc.) → add required and pattern validation attributes → tab through it using only the keyboard
3	**Semantic Blog Layout** — practice article-level structure **Steps:** mark up a blog post with article, time, and figure/figcaption → add a nested comments section → confirm heading levels (h1-h3) are in logical order
4	**ARIA-Enhanced Widget** — accessibility beyond plain HTML **Steps:** build a simple dropdown or accordion → add appropriate ARIA roles and aria-expanded → test it with a screen reader or accessibility dev tools → fix any issues the audit flags
5	**SEO Meta Tag Pass** — make a page shareable and indexable **Steps:** add title and meta description → add Open Graph tags for link previews → add a viewport meta tag → test the preview with a social share debugger tool

■ Study Resources

- MDN: HTML Basics — https://developer.mozilla.org/en-US/docs/Web/HTML
- MDN: Learn HTML — https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Structuring_content
- web.dev: Learn HTML — https://web.dev/learn/html

CSS

Controlling layout and visual presentation — the box model, modern layout systems, and responsive design.

Subtopics: *box model & positioning · Flexbox · Grid · responsive design/media queries · CSS variables & transitions*

1	**Responsive Navbar** — Flexbox in a real component **Steps:** build a horizontal navbar with Flexbox → add a media query collapsing it to a mobile menu → add a smooth transition on the mobile menu open/close → test at 3 different viewport widths
2	**CSS Grid Photo Gallery** — Grid for two-dimensional layout **Steps:** build a responsive grid with auto-fit/auto-fill → add a hover effect scaling each photo slightly → make the gallery reflow from 4 to 2 to 1 columns as the viewport shrinks
3	**Pricing Cards** — Flexbox for a common UI pattern **Steps:** build 3 pricing tier cards in a Flexbox row → make the middle "featured" card slightly larger → add a hover transition on each card → stack them vertically on mobile
4	**CSS Variables Theme Switcher** — practice custom properties **Steps:** define colors as CSS variables on :root → build a light and dark set of values → toggle a class on to switch themes → confirm every themed element updates instantly
5	**Box Model Debugging Drill** — the classic beginner trap, on purpose **Steps:** build a layout where padding/border unexpectedly break a fixed width → diagnose it using browser dev tools' box model view → fix it with box-sizing: border-box → explain in a comment why the bug happened

■ Study Resources

- MDN: CSS Reference — https://developer.mozilla.org/en-US/docs/Web/CSS
- CSS-Tricks: A Complete Guide to Flexbox — https://css-tricks.com/snippets/css/a-guide-to-flexbox/
- CSS-Tricks: A Complete Guide to Grid — https://css-tricks.com/snippets/css/complete-guide-grid/

Bootstrap / Tailwind CSS

One component-based framework and one utility-first framework, so you recognize both styles in real codebases and can pick the right one per project.

Subtopics: *Bootstrap grid & components · Tailwind utility classes & config · responsive breakpoint prefixes · when to choose each*

1	**Portfolio Rebuild — Bootstrap** — component-based styling **Steps:** rebuild your Phase 5 HTML portfolio using Bootstrap's grid → use prebuilt components (navbar, cards) instead of custom CSS → customize the theme colors via Bootstrap variables → time how long the rebuild took
2	**Portfolio Rebuild — Tailwind** — utility-first styling **Steps:** rebuild the same portfolio in Tailwind instead → use only utility classes, no custom CSS file → use responsive prefixes (sm:/md:/lg:) for breakpoints → compare the resulting HTML verbosity to the Bootstrap version

3	**Custom Tailwind Config** — go beyond the defaults **Steps:** extend tailwind.config with a custom color palette → add a custom font family → use your custom tokens in a component → confirm the build only includes classes actually used
4	**Bootstrap Modal + Form** — a common prebuilt-component workflow **Steps:** add a Bootstrap modal triggered by a button → put a form inside it → wire up basic client-side validation classes → confirm it's keyboard-accessible (Esc to close)
5	**Component Comparison Writeup** — reflect on the trade-off **Steps:** list 5 things that were faster in Bootstrap → list 5 things that were faster/cleaner in Tailwind → decide which you'd default to for ShopFlow's admin panel vs its public storefront

■ Study Resources

- Bootstrap Documentation — https://getbootstrap.com/docs/
- Tailwind CSS Documentation — https://tailwindcss.com/docs
- Tailwind CSS: Responsive Design — https://tailwindcss.com/docs/responsive-design

JavaScript (ES6+)

The language that actually runs in the browser — modern syntax, the DOM, and asynchronous programming.

Subtopics: *let/const, arrow functions, destructuring · DOM manipulation & events · Promises & async/await · closures & modules · array methods (map/filter/reduce)*

1	**Vanilla JS To-Do List** — DOM manipulation fundamentals **Steps:** render a list from a JS array → add an input + button to add new items → add click handlers to remove/complete items → persist nothing yet — pure in-memory state
2	**Quiz App** — state, events, and timing **Steps:** store questions/answers in an array of objects → render one question at a time → track score across questions → add a countdown timer using setInterval
3	**Weather App** — real async data fetching **Steps:** call a public weather API with fetch() → handle the loading state while waiting → handle the error state if the call fails → render the result once it resolves
4	**Array Methods Drill** — map/filter/reduce in isolation **Steps:** take an array of product objects → use map() to extract just the names → use filter() to find in-stock ones → use reduce() to compute the total inventory value
5	**Closures & Module Pattern** — understand scope deeply **Steps:** write a counter function using a closure to keep private state → confirm two separate counters don't interfere → split your code into ES modules with import/export → confirm the browser loads them correctly with type="module"

■ Study Resources

- MDN: JavaScript Guide — https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide
- javascript.info: The Modern JavaScript Tutorial — https://javascript.info/
- MDN: Using Promises — https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises

TypeScript

JavaScript with a type system layered on top — catching a whole category of bugs before the code ever runs.

Subtopics: *basic types & interfaces · union/intersection types · type narrowing · generics in TypeScript*

1	**To-Do List → TypeScript** — convert a JS project to TS **Steps:** add a Task interface (id, text, done) → type every function's parameters and return value → fix every type error the compiler reports → confirm the app still behaves identically
2	**Typed API Response Models** — type external data you don't control **Steps:** define an interface matching the weather API's JSON shape → type the fetch() result against it → add a union type for a field that can be one of several literal strings → confirm TypeScript catches a typo'd field name

3	**Discriminated Union Practice** — type narrowing on a shared "kind" field **Steps:** model 2-3 shape types sharing a kind field (circle/square) → write a function using a switch on kind → confirm TypeScript narrows the type correctly inside each case → add a new shape and let the compiler flag the missing switch case
4	**Generic Utility Function** — mirror the Java generics you already know **Steps:** write a generic function first(items: T[]): T → use it with arrays of different types → add a bounded generic requiring an id field → confirm the compiler enforces the bound
5	**Strict Mode Migration** — turn on the strictest settings and fix everything **Steps:** enable strict: true in tsconfig.json → fix every new error that appears → pay special attention to implicit any errors → confirm the project still builds cleanly after

■ Study Resources

- TypeScript Official Handbook — https://www.typescriptlang.org/docs/handbook/intro.html
- TypeScript: TSConfig Reference — https://www.typescriptlang.org/tsconfig
- Total TypeScript: Beginner's Tutorial (free) — https://www.totaltypescript.com/tutorials/beginners-typescript

Final project for this phase

ShopFlow — Static Storefront

- Product listing and detail pages in semantic HTML
- Fully responsive layout (Grid/Flexbox), styled with Bootstrap or Tailwind (your choice)
- Cart UI in vanilla TypeScript — add/remove, running total, localStorage persistence
- Fetch product data from a local JSON file (stand-in for the real API)
- No framework yet — proves you can build a real UI with just HTML/CSS/TS

This is your frontend fundamentals proof-of-concept, the same way Phase 1's console cart proved your OOP. Phase 6 rebuilds this in React and wires it to your real Spring Boot API.

PHASE 6 OF 10

Frontend Application Layer

React · Axios · API integration · Frontend dev tools

Now connect your frontend fundamentals to your real Spring Boot backend from Part A. Deployment for both the frontend and backend gets its own full track in Part C — this phase focuses purely on the application code and the tools you'll use to build it.

React + TypeScript

A component-based UI library, typed with TypeScript, for building the interface your users actually interact with.

Subtopics: *JSX, props, state, hooks · React Router (protected routes) · Context API & state management · form handling*

1	**Auth Pages** — login/register with route protection **Steps:** build a Login and Register form with controlled inputs → call your Phase 3 auth API on submit → store the returned JWT → build a ProtectedRoute wrapper redirecting unauthenticated users
2	**Product Listing Page** — fetch, filter, and display real data **Steps:** fetch products from your Spring Boot API → add category filter and search inputs → add a loading skeleton while fetching → handle the empty-results case gracefully
3	**Shopping Cart UI** — shared state across components **Steps:** build cart state in a Context provider → add quantity controls and a remove button per item → compute and display the running total → persist the cart across a page refresh
4	**Order History Page** — a protected, paginated view **Steps:** protect the route so only logged-in users can see it → fetch paginated orders from your API → click a row to expand order detail → handle the zero-orders empty state
5	**Reusable Component Library** — Button, Input, Modal, Table, Pagination **Steps:** extract a Button and Input component with typed props → extract a reusable Modal → extract a Table and Pagination component → use all five across at least 2 different pages

■ Study Resources

- React Official Documentation — https://react.dev/learn
- React Router Documentation — https://reactrouter.com/
- TypeScript: React Cheatsheet (react-typescript-cheatsheet) — https://react-typescript-cheatsheet.netlify.app/

Axios & API Integration

A dedicated HTTP client layered with interceptors, replacing scattered fetch() calls with one consistent, authenticated API client.

Subtopics: *Axios vs fetch · centralized instance & baseURL config · request/response interceptors · centralized error handling*

1	**Axios API Client Setup** — one instance, used everywhere **Steps:** create a single axios.create() instance with baseURL → set default headers and a timeout → replace every fetch() call in your app with it → confirm all API calls now go through one place
2	**JWT Attach Interceptor** — stop passing the token manually **Steps:** add a request interceptor reading the token from storage → attach it as an Authorization header automatically → confirm every request now includes it without manual code → test one request with no token stored
3	**Auto Token Refresh on 401** — the most valuable interceptor pattern **Steps:** add a response interceptor catching 401 errors → call your refresh-token endpoint from inside it → retry the original failed request with the new token → test it by using an intentionally expired token
4	**Centralized Error Shape** — consistent errors across the whole app **Steps:** add a response interceptor normalizing every error into one shape → surface network errors vs API errors distinctly → display a consistent toast/error message component → test both a 400 and a network-offline case
5	**Request Cancellation** — avoid race conditions on fast typing **Steps:** add an AbortController to a search-as-you-type request → cancel the previous request when a new one starts → confirm only the latest search result ever renders → test by typing quickly and watching the network tab

■ Study Resources

- Axios Documentation — https://axios-http.com/docs/intro
- Axios: Interceptors Guide — https://axios-http.com/docs/interceptors
- MDN: Using the Fetch API (for comparison) — https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

Types of APIs You'll Work With

Recognizing the different API styles you'll encounter on the job, beyond the REST/GraphQL you've already built.

Subtopics: *REST (recap) · GraphQL (recap) · WebSocket/Server-Sent Events · Webhooks · gRPC & SOAP (awareness only)*

1	**Live Order Status Widget (WebSocket)** — a persistent connection instead of polling **Steps:** connect to a WebSocket or SSE endpoint from React → subscribe to order-status update events → update the UI in real time as events arrive → confirm no polling/interval requests are happening
2	**Compare Polling vs WebSocket** — feel the difference directly **Steps:** build the same live-status feature with setInterval polling first → then rebuild it with WebSocket/SSE → compare network tab traffic between the two → write 3 bullet points on the trade-offs

3	**Webhook Receiver Endpoint** — the server side of an event push **Steps:** build a Spring Boot endpoint that accepts an incoming webhook POST → verify a signature header if the provider sends one → log the payload → return the expected 200 quickly to acknowledge receipt
4	**REST vs GraphQL Field Recap** — connect back to Phase 3's comparison **Steps:** revisit your Phase 3 REST vs GraphQL notes → call both from this React app → note any frontend-side difference in how you consume each → decide which you'd use for ShopFlow's product search
5	**API Style Spotting Exercise** — recognize what you're looking at in the wild **Steps:** find 3 public API docs online (any company) → identify whether each is REST, GraphQL, or something else → note anything that looks like gRPC or SOAP if you find it → write one sentence per API describing its style

■ Study Resources

- MDN: WebSockets API — https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API
- MDN: Server-Sent Events — https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events
- Spring: WebSocket Support Reference — https://docs.spring.io/spring-framework/reference/web/websocket.html

Frontend Dev Tools

The tools you'll reach for constantly while building the above — reference, not a project list.

Package Managers	npm, pnpm, or yarn — lockfiles and semantic versioning
Build Tool	Vite — dev server, fast HMR, production build config
Code Quality	ESLint + Prettier — consistent style, catching bugs before runtime
Browser DevTools	Network tab (inspect API calls), Console, Application tab (storage/cookies)
Framework DevTools	React Developer Tools browser extension
API Testing	Postman/Insomnia — test your Spring Boot endpoints before wiring up the frontend

■ Study Resources

- Vite Documentation — https://vite.dev/guide/
- ESLint Documentation — https://eslint.org/docs/latest/
- Chrome DevTools Documentation — https://developer.chrome.com/docs/devtools/

Final project for this phase

ShopFlow — React Frontend

- Rebuild the storefront in React + TypeScript, wired to the real Spring Boot API from Part A
- Centralized Axios client with interceptors for JWT attach + auto-refresh on 401
- Live order status using WebSocket/SSE instead of polling
- Reusable component library (Button, Input, Modal, Table, Pagination) used consistently
- Full auth flow: register, login, protected routes, logout

Deploying this frontend (and the Spring Boot backend behind it) to a live URL is covered in full in Part C — Deployment & Cloud, right after External APIs.

Part B complete. You now have a working React frontend talking to a real backend. Part C connects everything to the outside world: third-party services, and a live, deployed URL.

PART C

Deployment, Cloud & Integrations

Connecting to the outside world, and shipping ShopFlow to a live URL

PHASE 7 OF 10

External APIs

Third-party integration · Payment · Email · Auth providers · AI

Real apps never live in isolation. This phase connects ShopFlow to the services every production app depends on.

Payment Integration

Accepting real (or test-mode) payments, and handling the asynchronous confirmation that comes back after the charge.

Subtopics: *Stripe/PayMongo checkout flow · payment intents · webhook confirmation · idempotency*

1	**Stripe/PayMongo Checkout** — the core payment flow **Steps:** create a payment intent on the backend → confirm it on the frontend with the provider's SDK → handle the success and failure UI states → test with the provider's official test card numbers
2	**Webhook for Payment Confirmation** — why you can't trust the frontend alone **Steps:** build a webhook endpoint for the payment provider → verify the webhook signature → update the order status to PAID only from the webhook → test with the provider's webhook CLI/test tool
3	**Idempotent Webhook Handling** — handle the same event arriving twice **Steps:** simulate the same webhook event firing twice → add an idempotency check (e.g. by event ID) → confirm the order is only marked PAID once → log the duplicate as ignored, not as an error
4	**Failed Payment Handling** — the unhappy path matters more than the happy one **Steps:** trigger a declined test card → confirm the order stays PENDING, not PAID → surface a clear error message to the user → allow the user to retry with a different card
5	**Refund Flow** — closing the loop **Steps:** build an admin action to refund an order → call the provider's refund API → update the order status accordingly → confirm the webhook for the refund event is also handled

■ Study Resources

- Stripe Documentation — https://stripe.com/docs
- PayMongo Developer Documentation — https://developers.paymongo.com/docs
- Stripe: Webhooks Guide — https://stripe.com/docs/webhooks

Email Integration

Sending transactional emails triggered by real events in your app — not marketing email, just confirmations and receipts.

Subtopics: *SendGrid/Resend setup · transactional email templates · triggering on backend events · deliverability basics*

1	**Order Confirmation Email** — the most common transactional email **Steps:** set up SendGrid or Resend → build an HTML email template with the order summary → trigger it from your order-placed event → test it lands correctly in a real inbox
2	**Password Reset Email** — a security-sensitive flow **Steps:** generate a time-limited reset token → email a reset link containing it → build the reset-password page it links to → confirm the token expires and can't be reused
3	**Shipping Notification Email** — trigger from a status change, not a create **Steps:** trigger an email when order status changes to SHIPPED → include tracking info in the template → test triggering it via your admin panel's status update
4	**Email Template Variables** — keep templates DRY **Steps:** build one base template with variable placeholders → reuse it for both order confirmation and shipping notice → pass different variables per use case → confirm both render correctly
5	**Failed Email Handling** — don't let a broken inbox break checkout **Steps:** simulate the email provider being unreachable → confirm the order still completes successfully → log the email failure separately for retry → add a simple retry-once mechanism

■ Study Resources

- SendGrid Documentation — https://docs.sendgrid.com/
- Resend Documentation — https://resend.com/docs/introduction
- Baeldung: Sending Email with Spring — https://www.baeldung.com/spring-email

OAuth & Social Login

Letting users sign in with an existing Google/GitHub account instead of creating a new password, while still issuing your own JWT.

Subtopics: *OAuth 2.0 authorization code flow · exchanging a code for a profile · linking to your own user model · issuing your own JWT after OAuth*

1	**Google OAuth Login** — the full round trip **Steps:** register an OAuth app with Google → redirect users to Google's consent screen → exchange the returned code for a user profile → issue your own JWT on success, same as email/password login
2	**Account Linking** — handle a user who signs in both ways **Steps:** detect if the OAuth email matches an existing account → link the OAuth login to that existing user instead of creating a duplicate → test signing in via Google after previously registering by email

3	**New User via OAuth** — first-time OAuth sign-in **Steps:** detect a brand-new OAuth email → auto-create a user record from the OAuth profile data → redirect to a profile-completion step if needed → confirm the new user has a working JWT immediately
4	**OAuth Error Handling** — the user denies consent, or something fails **Steps:** simulate the user clicking "deny" on the consent screen → handle the resulting error redirect gracefully → show a clear message and a way to try again
5	**Second Provider: GitHub OAuth** — generalize the pattern **Steps:** add GitHub as a second OAuth provider → reuse as much of your Google flow's logic as possible → confirm both providers can link to the same account by email → note what had to change vs what was reusable

■ Study Resources

• Google Identity: OAuth 2.0 Documentation — https://developers.google.com/identity/protocols/oauth2
• GitHub: Authorizing OAuth Apps — https://docs.github.com/en/apps/oauth-apps/building-oauth-apps/authorizing-oauth-apps
• Spring Security: OAuth2 Client Reference — https://docs.spring.io/spring-security/reference/servlet/oauth2/index.html

File Storage & Uploads

Handling user-uploaded files (like product images) by streaming them to cloud storage instead of the app server's own disk.

Subtopics: *multipart form uploads · Cloudinary/S3 SDK basics · storing the URL not the file · validating file type/size*

1	**Product Image Upload** — the end-to-end flow **Steps:** build a multipart upload form in React → accept the file on a Spring Boot endpoint → upload it to Cloudinary (or S3) from the backend → store the returned URL on the Product entity
2	**Drag-and-Drop Upload UI** — a nicer upload experience **Steps:** replace the plain file input with a drag-and-drop zone → show a preview before upload → show upload progress → handle a drag of a non-image file gracefully
3	**File Validation** — reject bad uploads before they cost you storage **Steps:** validate file type on both frontend and backend → cap file size and reject oversized uploads → return a clear error message on rejection → test with an intentionally oversized file
4	**Image Transformation on Upload** — let the storage provider do the resizing **Steps:** configure an automatic thumbnail transformation on upload → store both the full and thumbnail URLs → use the thumbnail in the product list, full image on the detail page
5	**Delete Orphaned Files** — cleanup when a product is deleted **Steps:** delete the product record → also call the storage provider's delete API for its image → confirm no orphaned files are left behind → log any delete failures for manual cleanup

■ Study Resources

- Cloudinary Documentation — https://cloudinary.com/documentation
- AWS S3 Developer Guide — https://docs.aws.amazon.com/s3/
- Baeldung: Multipart File Upload with Spring — https://www.baeldung.com/spring-file-upload

AI API Integration

Calling a large language model API from your own backend to power a feature, not just chatting with one yourself.

Subtopics: *prompting from server-side code · structured/JSON responses · rate limits & cost awareness · graceful fallback*

1	**AI Product Description Generator** — the core integration pattern **Steps:** send a product name + category to the Claude or OpenAI API → request a generated marketing description → display it as a suggestion the admin can edit → handle the API call failing gracefully
2	**Structured JSON Response** — get parseable data back, not just prose **Steps:** prompt the API to return strict JSON (e.g. tags + description) → parse the response on the backend → validate the shape before saving it → handle malformed JSON without crashing
3	**AI-Powered Search Assistant** — a more advanced integration **Steps:** accept a natural-language search query → send it to the AI API to extract structured filters (category, price range) → run those filters against your real product search → return actual ShopFlow results, not AI-generated ones

4	**Rate Limit & Cost Guardrails** — don't let this feature blow up your bill **Steps:** add a per-user rate limit on the AI endpoint → cache repeated identical requests briefly → log token usage per call → add a hard daily cap with a friendly fallback message
5	**Fallback When AI Is Unavailable** — the feature should degrade, not break **Steps:** simulate the AI API being down or timing out → confirm the rest of the app still works → show a clear "suggestion unavailable" state instead of an error page

■ Study Resources

- Anthropic API Documentation — https://docs.claude.com
- OpenAI API Documentation — https://platform.openai.com/docs
- Anthropic: Prompt Engineering Overview — https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview

Final project for this phase

ShopFlow — Production-Ready Integrations

- PayMongo/Stripe real payment flow; webhook updates order to PAID
- Email: order confirmation + shipping notification templates
- Google OAuth 'Sign in with Google' alongside email/password
- Product image upload: drag-and-drop, stored on Cloudinary
- AI assistant: search bar powered by the Claude API

PHASE 8 OF 10

Deployment & Cloud

Docker · CI/CD · Cloud compute · Cloud storage · Hosting · Domains · Monitoring

Code that only runs on your laptop isn't a portfolio piece yet. This phase takes everything from Parts A and B and puts it on a real, public URL — the way it would actually run at a company.

Docker & Containerization

Packaging your app plus everything it needs to run into one portable unit that behaves identically everywhere.

Subtopics: *images vs containers · writing a Dockerfile · docker-compose for multi-service apps · volumes & networking basics*

1	**Dockerize Spring Boot** — the core backend containerization workflow **Steps:** write a Dockerfile for your Spring Boot app → build the image with docker build → run it with docker run and hit the API → confirm it works identically to running it locally with Maven
2	**Dockerize the React Frontend** — a static-build container **Steps:** write a multi-stage Dockerfile (build stage + nginx serve stage) → build and run the image → confirm the built frontend serves correctly on the exposed port
3	**docker-compose for the Full Stack** — run everything with one command **Steps:** write a docker-compose.yml with backend, frontend, and PostgreSQL services → configure networking so they can reach each other by service name → run docker-compose up and confirm the full stack works together
4	**Environment Variables in Docker** — keep secrets out of the image itself **Steps:** move DB credentials and JWT secret to environment variables → pass them via docker-compose's environment section or an .env file → confirm the image itself contains no hardcoded secrets
5	**Volume for Persistent Data** — don't lose your database on container restart **Steps:** add a named volume for PostgreSQL's data directory → stop and remove the container → start a new one against the same volume → confirm your data is still there

■ Study Resources

- Docker: Get Started Guide — https://docs.docker.com/get-started/
- Docker: Dockerfile Reference — https://docs.docker.com/reference/dockerfile/
- Docker Compose Documentation — https://docs.docker.com/compose/

CI/CD with GitHub Actions

Automatically testing and building your code on every push, so broken code never gets the chance to reach production silently.

Subtopics: *workflow YAML basics · running tests on push · building & pushing a Docker image · deploying on merge to main*

1	**First GitHub Actions Workflow** — the minimal CI pipeline **Steps:** add a .github/workflows/ci.yml file → trigger it on push and pull_request → run mvn test (or npm test) in it → confirm it shows a red X on a failing test, green check on passing
2	**Multi-Job Pipeline** — backend and frontend tested separately **Steps:** split the workflow into a backend-test job and a frontend-test job → run them in parallel → confirm both must pass before the workflow succeeds
3	**Build & Push a Docker Image** — extend CI into CI/CD **Steps:** add a job that builds your Docker image on merge to main → push it to a container registry (Docker Hub or GHCR) → tag it with the commit SHA → confirm the image appears in the registry
4	**Require CI to Pass Before Merge** — connect this back to branch protection **Steps:** go to your repo's branch protection settings → require this workflow to pass before merging to main → confirm a PR with a failing test literally cannot be merged
5	**Auto-Deploy on Merge** — the final link in the chain **Steps:** add a deploy job triggered only on push to main → have it call your hosting provider's deploy hook or CLI → confirm a merge to main results in a live update automatically

■ Study Resources

- GitHub Actions Documentation — https://docs.github.com/en/actions
- GitHub Actions: Building and Testing Java with Maven — https://docs.github.com/en/actions/use-cases-and-examples/building-and-testing/building-and-testing-java-with-maven
- GitHub Actions: Workflow Syntax Reference — https://docs.github.com/en/actions/reference/workflow-syntax-for-github-actions

Cloud Compute Basics

Where your backend actually runs once it's not localhost — awareness of the major providers, hands-on with at least one.

Subtopics: *AWS EC2/Elastic Beanstalk basics · Google Cloud Run/App Engine awareness · Azure App Service awareness · choosing compute vs PaaS*

1	**Launch a Basic EC2 Instance** — the fundamental building block of AWS compute **Steps:** launch a small free-tier EC2 instance → SSH into it → install Java and run your Spring Boot JAR manually on it → hit the API from your own machine over the instance's public IP
2	**Deploy via Elastic Beanstalk** — a more managed AWS option **Steps:** package your Spring Boot app for Elastic Beanstalk → deploy it through the EB CLI or console → confirm it's running behind a managed load balancer → compare the setup effort to raw EC2

3	**Google Cloud Run Deployment** — a serverless-container alternative **Steps:** build your Docker image from the earlier topic → deploy it to Cloud Run → confirm it scales to zero when idle → compare cost/complexity to EC2
4	**Compute Provider Comparison** — reflect on what you tried **Steps:** list what you used from AWS, GCP, or Azure → note setup time, pricing model, and ease of use for each → decide which you'd default to for a personal project vs a company project
5	**Shut Everything Down** — the most commonly skipped but critical step **Steps:** terminate/stop every cloud compute resource you created in this topic → confirm in the billing dashboard that nothing is still running → make this a habit after every cloud experiment from here on

■ Study Resources

- AWS: EC2 User Guide — https://docs.aws.amazon.com/ec2/
- AWS: Elastic Beanstalk Developer Guide — https://docs.aws.amazon.com/elasticbeanstalk/
- Google Cloud Run Documentation — https://cloud.google.com/run/docs

Cloud Storage

Storing files (images, documents, backups) outside your application server, in a service built to hold them reliably at any scale.

Subtopics: *AWS S3 buckets & objects · Google Cloud Storage basics · Cloudinary (media-focused storage) · public vs private access & signed URLs*

1	**Create and Use an S3 Bucket** — the fundamental cloud storage workflow **Steps:** create an S3 bucket via the AWS console → upload a file manually to confirm it works → upload a file programmatically from your Spring Boot app using the AWS SDK → retrieve it back via its URL
2	**Public vs Private Bucket Access** — understand access control before you get it wrong **Steps:** configure a bucket as private by default → generate a signed/pre-signed URL with a short expiry → confirm the file is NOT accessible without the signed URL → confirm it IS accessible through it, until it expires
3	**Google Cloud Storage Equivalent** — compare the second-most-common option **Steps:** create a GCS bucket → upload and retrieve a file from your backend → compare the SDK/API differences from S3 → note which felt simpler to set up
4	**Cloudinary for Media-Specific Needs** — when you want more than raw storage **Steps:** revisit your Phase 7 Cloudinary upload integration → add an automatic image transformation on upload (resize/format) → compare this to plain S3, which does no processing for you → decide which you'd use for product images vs generic file storage
5	**Storage Cost & Lifecycle Awareness** — cloud storage isn't free forever **Steps:** read your provider's pricing page for storage + egress → set up a lifecycle rule to auto-delete files after N days in a test bucket → confirm the rule is active → delete all test buckets when done

■ Study Resources

- AWS S3 Developer Guide — https://docs.aws.amazon.com/s3/
- Google Cloud Storage Documentation — https://cloud.google.com/storage/docs
- Cloudinary Documentation — https://cloudinary.com/documentation

PaaS Hosting (Vercel, Netlify, Railway, Render)

The fastest path from GitHub repo to live URL — platform-as-a-service providers that handle the server management for you.

Subtopics: *frontend hosting (Vercel/Netlify) · backend hosting (Railway/Render) · auto-deploy on push · preview URLs per PR*

1	**Deploy the Frontend to Vercel/Netlify** — the fastest possible deploy **Steps:** connect your GitHub repo to Vercel or Netlify → configure the build command and output directory → confirm auto-deploy triggers on push to main → open a PR and confirm a preview URL is generated
2	**Deploy the Backend to Railway/Render** — PaaS for your Spring Boot API **Steps:** connect your backend repo (or push the Docker image) → configure environment variables in the platform's dashboard → confirm the API is reachable at its generated URL → connect it to a managed PostgreSQL instance from the same platform

3	**Environment-Based API URLs** — don't hardcode localhost anywhere **Steps:** add a build-time env variable for the API base URL → confirm the deployed frontend calls the deployed backend, not localhost → confirm your local dev setup still points at localhost
4	**Preview Environments for PRs** — test changes before they're live **Steps:** open a PR with a small visible change → confirm a unique preview URL is generated for it → share that link as if reviewing someone else's work → merge and confirm it promotes to production
5	**Compare PaaS vs Cloud Compute** — connect this back to the previous topic **Steps:** list what Railway/Render/Vercel handled automatically → list what you had to do manually on EC2 earlier → decide which you'd pick for ShopFlow specifically, and why

■ Study Resources

- Vercel Documentation — https://vercel.com/docs
- Netlify Documentation — https://docs.netlify.com/
- Railway Documentation — https://docs.railway.com/

Domains & HTTPS

Pointing a real domain name at your deployed app, and making sure every connection to it is encrypted.

Subtopics: *DNS records (A/CNAME) · connecting a custom domain · HTTPS & SSL certificates · why HTTPS is non-negotiable*

1	**Point a Domain at Vercel/Netlify** — the standard frontend DNS setup **Steps:** buy or use a free-tier domain → add the CNAME/A record your hosting provider specifies → wait for DNS propagation → confirm your domain loads the deployed frontend
2	**Subdomain for the API** — separate frontend and backend domains cleanly **Steps:** add a CNAME for api.yourdomain.com pointing at your backend host → update your frontend's API base URL to use it → confirm CORS is configured to allow the frontend domain
3	**Confirm Automatic HTTPS** — most PaaS providers do this for you now **Steps:** confirm your domain shows a valid padlock/certificate in the browser → inspect the certificate details → explain in a comment who issued it and how it auto-renews
4	**DNS Record Types Drill** — understand what you actually configured **Steps:** look up your domain's A, CNAME, and (if any) MX records → explain what each one does → explain why an A record and a CNAME can't both target the same name at the root
5	**Force HTTPS Redirect** — close the insecure-access gap **Steps:** attempt to load your site over plain http:// → confirm it force-redirects to https:// → if it doesn't by default, configure the redirect explicitly

■ Study Resources

- MDN: What is a Domain Name? — https://developer.mozilla.org/en-US/docs/Learn_web_development/Howto/Web_mechanics/What_is_a_domain_name
- Cloudflare Learning Center: DNS Records — https://www.cloudflare.com/learning/dns/dns-records/
- MDN: What is HTTPS? — https://developer.mozilla.org/en-US/docs/Glossary/HTTPS

Monitoring & Error Tracking

Finding out about production problems from a dashboard, not from an angry user — the last mile of shipping software responsibly.

Subtopics: *uptime monitoring · error tracking (Sentry) · basic application logging in production · alerting*

1	**Uptime Monitor Setup** — know the moment your site goes down **Steps:** sign up for a free uptime monitoring service → point it at your deployed frontend and backend URLs → configure an email/notification alert on downtime → test it by intentionally stopping the backend briefly
2	**Sentry Error Tracking** — catch exceptions you'd otherwise never see **Steps:** add Sentry (or a similar tool) to both frontend and backend → trigger a deliberate error in each → confirm it appears in the Sentry dashboard with a full stack trace → add user context to make errors easier to diagnose

3	**Production Logging Review** — connect this back to Phase 2's Logging topic **Steps:** confirm your SLF4J logs are visible in your hosting provider's log viewer → search/filter them for a specific request → confirm no sensitive data is appearing in production logs
4	**Basic Alerting Rule** — get notified, don't just have data sitting there **Steps:** configure an alert for a spike in error rate (not just total downtime) → test it by triggering several errors in a short window → confirm the alert fires as expected
5	**Incident Postmortem Practice** — the habit that separates junior from senior responses **Steps:** pick any bug you fixed earlier in this roadmap → write a short postmortem: what happened, why, what you changed → note one monitoring/alerting improvement that would have caught it sooner

■ Study Resources

- Sentry Documentation — https://docs.sentry.io/
- UptimeRobot (free uptime monitoring) — https://uptimerobot.com/
- Grafana: Getting Started (for later, more advanced monitoring) — https://grafana.com/docs/grafana/latest/getting-started/

Final project for this phase

ShopFlow — Fully Deployed & Live

- Backend containerized with Docker, deployed to Railway/Render, connected to hosted PostgreSQL + MongoDB
- Frontend deployed to Vercel/Netlify with environment-based API URLs (dev vs prod)
- GitHub Actions pipeline running tests on every push, deploying automatically on merge to main
- Product images stored in S3 or Cloudinary, not on the app server
- Custom domain with HTTPS pointing at the deployed frontend, api. subdomain for the backend
- Sentry (or similar) catching real errors, uptime monitoring alerting you if it goes down

This is the milestone that turns ShopFlow from "code on my laptop" into "a product with a URL." Everything from Parts A, B, and this phase now lives together, live, on the internet.

Part C complete. ShopFlow is now deployed, monitored, and integrated with real third-party services. Part D shifts to a different kind of skill entirely: data structures and algorithms.

PART D

Data Structures & Algorithms

The CS fundamentals every technical interview tests

PHASE 9 OF 10

Data Structures & Algorithms

Arrays · Trees · Graphs · Sorting · Dynamic programming

DSA last is the right call for your goals. You now have real context for why these matter — you've seen N+1 queries (a graph traversal problem), sorted product lists (sorting algorithms), and 2D arrays (Phase 2). Study in Java. Implement each data structure from scratch before using built-ins.

Arrays & Strings

The most fundamental data structure, and the pattern toolkit (two pointers, sliding window) that solves a huge share of interview questions.

Subtopics: *two pointers · sliding window · prefix sums · StringBuilder & string manipulation*

1	**Two Sum (and variants)** — the canonical two-pointer/hash problem **Steps:** solve it with a brute-force O(n²) approach first → solve it again with a HashMap in O(n) → solve the sorted-array version with two pointers → compare all three approaches' time complexity
2	**Sliding Window Maximum Substring** — the sliding window pattern **Steps:** find the longest substring without repeating characters → use a window with two pointers and a Set → track the max length as the window grows/shrinks → test with edge cases: empty string, all-same characters
3	**Prefix Sum Range Queries** — answer range-sum questions in O(1) after O(n) setup **Steps:** build a prefix sum array from an input array → answer "sum from index i to j" queries in O(1) → compare to recomputing the sum each time → apply it to a "subarray sums to target" problem
4	**String Reversal & Palindrome Check** — StringBuilder and two-pointer string basics **Steps:** reverse a string using StringBuilder → reverse it again using two pointers in-place on a char array → check if a string is a palindrome using two pointers → handle spaces/punctuation/case in the palindrome check
5	**Merge Intervals** — a very common array-of-pairs pattern **Steps:** sort intervals by start time → merge overlapping intervals in one pass → return the minimal merged list → test with intervals that are fully contained within others

■ Study Resources

- GeeksforGeeks: Array Data Structure — https://www.geeksforgeeks.org/dsa/array-data-structure-guide/
- NeetCode: Arrays & Hashing (free roadmap) — https://neetcode.io/roadmap
- Oracle: Arrays (Java Tutorials) — https://docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html

Linked Lists

A chain of nodes where insertion/deletion is cheap but random access isn't — and the classic source of pointer-manipulation interview questions.

Subtopics: *singly & doubly linked lists · fast/slow pointer technique · reversing a list · cycle detection*

1	**Build a Singly Linked List From Scratch** — no built-ins allowed **Steps:** write a Node class with value and next → implement add, remove, and get(index) → implement a toString() for easy debugging → test insertion at head, middle, and tail
2	**Reverse a Linked List** — the single most common linked-list question **Steps:** reverse it iteratively with 3 pointers (prev/curr/next) → reverse it again recursively → compare both approaches' space complexity → test on an empty list and a 1-node list
3	**Detect a Cycle (Floyd's Algorithm)** — the fast/slow pointer technique **Steps:** build a list with a deliberate cycle → implement fast/slow pointers to detect it → extend it to find where the cycle begins → test on a list with no cycle to confirm no false positive
4	**Doubly Linked List Implementation** — add a prev pointer **Steps:** extend your Node with a prev reference → implement insertion/removal from both ends in O(1) → implement forward and backward traversal → use this as the base for Part D's later LRU Cache problem
5	**Merge Two Sorted Linked Lists** — a common building block for merge sort on lists **Steps:** merge two already-sorted linked lists into one sorted list → do it without creating new nodes, just relinking → test with lists of different lengths → test with one empty input list

■ Study Resources

- GeeksforGeeks: Linked List Data Structure — https://www.geeksforgeeks.org/dsa/linked-list-data-structure/
- NeetCode: Linked List Roadmap Section — https://neetcode.io/roadmap
- Baeldung: Introduction to Linked Lists in Java — https://www.baeldung.com/java-linkedlist

Stacks & Queues

LIFO and FIFO structures underlying everything from undo buttons to breadth-first search.

Subtopics: *stack (push/pop/peek) · queue (enqueue/dequeue) · monotonic stack · implementing both from scratch*

1	**Stack From Scratch** — array-backed, no java.util.Stack **Steps:** implement push/pop/peek/isEmpty backed by a resizable array → handle the empty-pop case with a clear exception → test push/pop order matches LIFO expectations
2	**Valid Parentheses** — the classic stack interview question **Steps:** use a stack to match opening/closing brackets → handle 3 bracket types at once ( ) [ ] { } → return false on any mismatch or leftover open bracket → test with nested and malformed inputs
3	**Queue From Scratch** — array or linked-list backed **Steps:** implement enqueue/dequeue/peek/isEmpty → decide array vs linked-list backing and justify it in a comment → test FIFO order is correct

4	**Monotonic Stack: Next Greater Element** — a less obvious but common stack pattern **Steps:** for each element, find the next element to its right that's greater → solve it in O(n) using a monotonic decreasing stack → compare to the brute-force O(n²) approach → test with a strictly decreasing input (no next-greater exists)
5	**Implement a Queue Using Two Stacks** — a classic "combine two structures" exercise **Steps:** implement enqueue/dequeue using only two Stack instances internally → figure out which operations need to move elements between stacks → test the amortized cost stays reasonable

■ Study Resources

- GeeksforGeeks: Stack Data Structure — https://www.geeksforgeeks.org/dsa/stack-data-structure/
- GeeksforGeeks: Queue Data Structure — https://www.geeksforgeeks.org/dsa/queue-data-structure/
- NeetCode: Stack Roadmap Section — https://neetcode.io/roadmap

Trees

Hierarchical, recursive structures — the shape behind file systems, category menus, and JSON itself.

Subtopics: *binary tree traversals (in/pre/post/BFS) · binary search tree operations · tree height & balance · lowest common ancestor*

1	**Binary Tree Traversals** — the four fundamental ways to visit a tree **Steps:** implement in-order traversal recursively → implement pre-order and post-order recursively → implement level-order (BFS) using a Queue → print all four for the same tree and compare output
2	**Binary Search Tree From Scratch** — insert, search, delete **Steps:** implement insert() maintaining BST ordering → implement search() in O(log n) on a balanced tree → implement delete() handling all 3 cases (leaf, one child, two children) → implement an in-order iterator confirming sorted output
3	**Tree Height & Balance Check** — a very common recursive-tree warmup **Steps:** write a recursive height(node) function → write isBalanced() checking every subtree's height difference → test on a balanced tree and a deliberately skewed one
4	**Lowest Common Ancestor** — a classic recursive tree question **Steps:** implement LCA for a binary search tree using ordering properties → implement it again for a general binary tree (no ordering to rely on) → compare the two approaches → test with one node being an ancestor of the other
5	**Serialize & Deserialize a Binary Tree** — a harder but very common interview question **Steps:** design a string format encoding the tree (including nulls) → implement serialize() producing that string → implement deserialize() rebuilding the exact same tree → confirm round-tripping produces an identical tree

■ Study Resources

- GeeksforGeeks: Tree Data Structure — https://www.geeksforgeeks.org/dsa/tree-data-structure/
- NeetCode: Trees Roadmap Section — https://neetcode.io/roadmap
- Baeldung: Binary Tree in Java — https://www.baeldung.com/java-binary-tree

Heaps & Priority Queues

A tree-shaped structure optimized for one thing: instantly grabbing the smallest (or largest) item.

Subtopics: *min-heap vs max-heap · heapify (sift up/down) · top-K problems · heap sort*

1	**Min-Heap From Scratch** — array-backed, no PriorityQueue built-in **Steps:** implement insert() with sift-up → implement extractMin() with sift-down → implement peek() → test the heap property holds after several insert/extract cycles
2	**Kth Largest Element** — the signature top-K heap problem **Steps:** maintain a min-heap of size K while scanning the input → return the heap's root as the Kth largest at the end → compare this to fully sorting the array first → test with K equal to the array length

3	**Heap Sort** — an O(n log n) sort using your own heap **Steps:** build a max-heap from an unsorted array → repeatedly extract the max and place it at the end → confirm the result is fully sorted → compare its behavior to Java's built-in sort on the same input
4	**Merge K Sorted Lists** — a classic heap application **Steps:** put the head of each list into a min-heap → repeatedly extract the min and advance that list → build the merged result → test with lists of very different lengths
5	**Running Median with Two Heaps** — a harder but common heap pattern **Steps:** maintain a max-heap for the lower half and a min-heap for the upper half → keep them balanced as numbers stream in → return the median in O(1) at any point → test with an even and an odd number of elements seen so far

■ Study Resources

• GeeksforGeeks: Heap Data Structure — https://www.geeksforgeeks.org/dsa/heap-data-structure/
• Oracle: PriorityQueue (Javadoc, for comparison after building your own) — https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/PriorityQueue.html
• NeetCode: Heap/Priority Queue Roadmap Section — https://neetcode.io/roadmap

Hashing

Trading memory for near-constant-time lookups — understanding what's actually happening inside every HashMap you've used since Phase 2.

Subtopics: *HashMap internals (buckets & hashing) · collision handling · frequency maps · the two-sum family of problems*

1	**Implement a Basic HashMap** — understand what put()/get() actually do **Steps:** back it with an array of buckets → implement hashCode-based bucket selection → handle collisions with chaining (a linked list per bucket) → test with intentionally colliding keys
2	**Frequency Map Drill** — the most common hashing use case **Steps:** count character frequency in a string → count word frequency in a sentence → find the most frequent element in an array → solve "first non-repeating character" using the same technique
3	**Two Sum Family** — revisit with hashing specifically in focus **Steps:** solve two-sum with a HashMap in one pass → solve three-sum by fixing one element and hashing the rest → compare time complexity to the brute-force versions
4	**Group Anagrams** — hashing with a computed key **Steps:** group a list of words into anagram sets → use the sorted-characters string as the hash key → return groups as a List> → test with words that share no anagrams
5	**Longest Consecutive Sequence** — O(n) with a HashSet, not sorting **Steps:** put all numbers into a HashSet → for each number, check if it's the start of a sequence → extend forward counting consecutive numbers → confirm this beats the O(n log n) sort-based approach

■ Study Resources

- GeeksforGeeks: Hashing Data Structure — https://www.geeksforgeeks.org/dsa/hashing-data-structure/
- Oracle: HashMap (Javadoc) — https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/HashMap.html
- NeetCode: Arrays & Hashing Roadmap Section — https://neetcode.io/roadmap

Graphs

Nodes and edges modeling relationships — from social networks to delivery routes to your own N+1 query problem from Part A.

Subtopics: *adjacency list vs matrix · BFS & DFS · topological sort · Dijkstra's algorithm · union-find*

1	**Graph Representation & Traversal** — the foundation everything else builds on **Steps:** build a graph using an adjacency list (Map>) → implement BFS returning visit order → implement DFS (iterative with a stack, and recursive) → test on a graph with a disconnected component
2	**Connected Components** — DFS applied to a practical question **Steps:** given an undirected graph, count the number of connected components → use DFS or union-find to group nodes → test on a graph that's fully connected vs fully disconnected
3	**Topological Sort** — ordering with dependencies **Steps:** model a set of tasks with prerequisites as a directed graph → implement topological sort via DFS or Kahn's algorithm (BFS + in-degree) → detect and report a cycle (impossible ordering) if one exists

4	**Dijkstra's Shortest Path** — weighted graph pathfinding **Steps:** model a weighted graph (e.g. a delivery route network) → implement Dijkstra using a priority queue → return the shortest distance to every node from a source → test against a graph where the "obvious" path isn't shortest
5	**Union-Find (Disjoint Set)** — an alternative to DFS for connectivity questions **Steps:** implement union() and find() with path compression → use it to detect a cycle in an undirected graph → use it to count connected components again, compare to your DFS version

■ Study Resources

- GeeksforGeeks: Graph Data Structure and Algorithms — https://www.geeksforgeeks.org/dsa/graph-data-structure-and-algorithms/
- NeetCode: Graphs Roadmap Section — https://neetcode.io/roadmap
- Baeldung: Graph Theory in Java — https://www.baeldung.com/java-graphs

Sorting & Searching

The algorithms behind every ORDER BY and every binary search — and the ones interviewers expect you to implement, not just call.

Subtopics: *merge sort · quick sort · binary search & its variants · time complexity trade-offs*

1	**Merge Sort From Scratch** — a stable O(n log n) sort **Steps:** implement the recursive divide step → implement the merge step combining two sorted halves → test on an already-sorted and a reverse-sorted array → confirm it's stable (equal elements keep relative order)
2	**Quick Sort From Scratch** — an in-place O(n log n) average-case sort **Steps:** implement the partition step (Lomuto or Hoare) → implement the recursive quicksort around it → test worst-case behavior on an already-sorted array → discuss why pivot choice matters in a comment
3	**Binary Search & Its Variants** — beyond the textbook version **Steps:** implement standard binary search for an exact match → implement "find first occurrence" in a sorted array with duplicates → implement "find insertion point" for a value not present → test all three against edge cases (empty array, single element)
4	**Search in a Rotated Sorted Array** — a very common binary-search variant question **Steps:** given a sorted array rotated at an unknown pivot, find a target in O(log n) → figure out which half is properly sorted at each step → test with the target being the pivot element itself
5	**Sorting Algorithm Comparison** — reflect on what you built **Steps:** time your merge sort and quick sort on the same large random array → time Java's built-in Arrays.sort() on the same data → explain any difference you observe → note which algorithm you'd pick for nearly-sorted data

> **■ Study Resources**
>
> • GeeksforGeeks: Sorting Algorithms — https://www.geeksforgeeks.org/dsa/sorting-algorithms/
> • GeeksforGeeks: Binary Search — https://www.geeksforgeeks.org/dsa/binary-search/
> • NeetCode: Binary Search Roadmap Section — https://neetcode.io/roadmap

Dynamic Programming

Solving a big problem by breaking it into overlapping smaller subproblems and never solving the same one twice.

Subtopics: *memoization vs tabulation · knapsack pattern · longest common subsequence · coin change / unbounded knapsack*

1	**Fibonacci: Naive → Memoized → Tabulated** — the clearest possible DP progression **Steps:** implement naive recursive Fibonacci and time it for n=35 → add memoization with a HashMap or array cache → rewrite it bottom-up with tabulation, no recursion → compare all three approaches' time and space
2	**0/1 Knapsack** — the canonical DP pattern **Steps:** given items with weight/value and a capacity, maximize value → solve with a 2D DP table (items x capacity) → trace back which items were chosen → test with a capacity that fits everything vs almost nothing

3	**Longest Common Subsequence** — a classic 2-string DP problem **Steps:** build a 2D DP table comparing two strings → return the length of the longest common subsequence → extend it to reconstruct the actual subsequence, not just its length → test with strings that share nothing in common
4	**Coin Change (Unbounded Knapsack)** — minimum coins to make an amount **Steps:** solve for the minimum number of coins to reach a target amount → handle the impossible case (return -1 or similar) → solve the count-the-ways variant as a follow-up → compare unbounded (reuse allowed) vs 0/1 knapsack's constraint
5	**DP on a Grid: Unique Paths** — a 2D-array-flavored DP problem, tying back to Phase 2 **Steps:** count the number of unique paths from top-left to bottom-right of a grid, moving only right/down → solve with a 2D DP table → add obstacles that block certain cells → confirm the obstacle version still produces correct counts

■ Study Resources

- GeeksforGeeks: Dynamic Programming — https://www.geeksforgeeks.org/dsa/dynamic-programming/
- NeetCode: Dynamic Programming Roadmap Section — https://neetcode.io/roadmap
- Baeldung: Dynamic Programming in Java — https://www.baeldung.com/java-dynamic-programming

Final project for this phase

DSA Inside ShopFlow — Practical Applications

- Product search: implement trie-based autocomplete for search bar
- Category tree: recursive tree traversal to build nested category menu
- Order queue: priority queue (heap) for order processing by urgency/SLA
- Recommendation: graph of 'users who bought X also bought Y' (BFS)
- Inventory restock: min-heap to surface lowest-stock items first
- Delivery routing: Dijkstra on a city graph for shortest delivery path

Apply your DSA knowledge to real features in your existing ShopFlow project. This ties the abstract concepts to code you already understand.

Part D complete. You now have the CS fundamentals most self-taught developers skip. Part E closes the roadmap with everything that turns finished code into an actual offer.

PART E

Career & Interview Readiness

Turning finished ShopFlow work into an actual offer

PHASE 10 OF 10

Career & Interview Readiness

System design basics · Security basics · Resume · Interview prep

Technical skill alone doesn't get you the internship or offer — it gets you through the door. This closing phase covers what turns finished ShopFlow work into an actual application people will hire you off of.

System Design Basics

Not full distributed-systems design — just enough to talk intelligently about how ShopFlow, or any app, would need to change at scale.

Subtopics: *client-server & load balancing · caching (Redis) · database scaling (read replicas, sharding awareness) · rate limiting at scale*

1	**Draw ShopFlow's Current Architecture** — start from what you actually built **Steps:** diagram every piece: frontend, backend, PostgreSQL, MongoDB, external APIs → label where auth happens, where caching could go → practice explaining it out loud in under 2 minutes → record yourself and listen back once
2	**Add a Caching Layer (Redis)** — the single highest-leverage scaling addition **Steps:** identify ShopFlow's most-read, rarely-changed data (e.g. product catalog) → add Redis and cache that data → implement cache invalidation on product update → measure response time with and without the cache
3	**Design: "What if ShopFlow had 1 million users?"** — the classic scaling interview prompt **Steps:** identify the first component that would break → propose a fix (read replica, caching, queue, CDN) → repeat for the next bottleneck → write your answer as if explaining it in an interview
4	**Load Balancer Simulation** — understand horizontal scaling conceptually **Steps:** run 2 instances of your backend locally on different ports → put a simple reverse proxy (e.g. nginx) in front distributing requests → confirm requests are being split between both instances → explain what this buys you over one bigger instance
5	**Rate Limiting at Scale** — revisit Phase 3's rate limiter with a scaling lens **Steps:** review your Phase 3 per-IP rate limiter → explain why an in-memory rate limiter breaks with multiple backend instances → propose a fix (shared store like Redis for the rate limit counters) → implement it if time allows

■ Study Resources

- System Design Primer (free, community-maintained) — https://github.com/donnemartin/system-design-primer
- Redis Documentation — https://redis.io/docs/latest/
- AWS: What is Load Balancing? — https://aws.amazon.com/what-is/load-balancing/

Security Basics

The vulnerabilities every technical interviewer expects a backend developer to at least recognize by name.

Subtopics: *OWASP Top 10 awareness · input validation (never trust the client) · CORS · HTTPS, cookies & password hashing · secrets management*

1	**OWASP Top 10 Self-Audit** — the single most valuable security exercise here **Steps:** read through the current OWASP Top 10 → go through ShopFlow checking for each category → fix at least 3 real findings → document what you found and fixed
2	**SQL Injection Test (and confirm you're safe)** — prove your JPA/PreparedStatement usage protects you **Steps:** attempt a classic injection payload against a ShopFlow search field → confirm it fails harmlessly thanks to PreparedStatement/JPA → find any raw-concatenated query anywhere in your codebase and fix it if so
3	**CORS Configuration Review** — not just "allow all" **Steps:** review your current CORS configuration → confirm it only allows your actual frontend origin(s) → test that a request from a random origin is correctly blocked → explain in a comment what CORS does and doesn't protect against
4	**Password Storage Audit** — confirm you never stored anything in plaintext **Steps:** confirm passwords are hashed with BCrypt (or Argon2), never stored raw → confirm you can't "recover" a password, only reset it → check your logs for any accidental password/token logging
5	**Secrets Management Cleanup** — closing the loop from Part 0's Git section **Steps:** scan your repo history for any accidentally committed secret → rotate any credential that was ever exposed, even briefly → confirm all current secrets live in environment variables, not code

■ Study Resources

- OWASP Top 10 — https://owasp.org/www-project-top-ten/
- MDN: Web Security — https://developer.mozilla.org/en-US/docs/Web/Security
- MDN: Cross-Origin Resource Sharing (CORS) — https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS

Resume & Portfolio

Presenting a year of real work so a recruiter or hiring manager understands its value in the 30 seconds they'll actually spend on it.

Subtopics: *GitHub profile hygiene · impact-framed resume bullets · a portfolio site · live demo links*

1	**GitHub Profile Cleanup** — your profile is often checked before your resume **Steps:** pin your 4-6 best repos, ShopFlow first → write a proper README for each with a screenshot and live demo link → add a profile README introducing yourself → remove or archive genuinely abandoned experiments

2	**Resume Bullet Rewrite** — impact over task lists **Steps:** take 5 things you built this year → rewrite each as an impact statement, not a task description → quantify where honestly possible (endpoints, test coverage, response time) → get one outside opinion on the rewritten version
3	**Build Your Portfolio Site** — you already have every skill needed for this **Steps:** use your Part B skills to build a personal site → feature ShopFlow prominently with a live link and screenshots → add a short case study: problem, approach, stack, what you'd improve → deploy it using Part C's deployment skills
4	**Live Demo Checklist** — confirm everything a stranger clicks actually works **Steps:** open ShopFlow's live URL in a private/incognito window → walk through signup → browse → cart → checkout as a stranger would → fix anything broken or confusing → add a demo account so reviewers don't need to register
5	**LinkedIn / Professional Profile Pass** — often the first thing a recruiter actually opens **Steps:** update your headline and About section to reflect this year's work → add ShopFlow as a featured project with the live link → list your real stack accurately, not aspirationally

■ Study Resources

• GitHub Docs: About READMEs — https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes
• GitHub Docs: Managing your profile README — https://docs.github.com/en/account-and-profile/setting-up-and-managing-your-github-profile/customizing-your-profile/managing-your-profile-readme
• freeCodeCamp: How to Build a Portfolio — https://www.freecodecamp.org/news/tag/portfolio/

Interview Preparation

Turning a year of study into performance under real interview conditions — technical and behavioral.

Subtopics: *LeetCode practice cadence · STAR method for behavioral questions · mock interviews · matching your skills to job postings*

1	**Build a LeetCode Cadence** — consistency beats cramming **Steps:** pick a realistic weekly target (e.g. 3-5 problems) → focus on patterns from Part D you're weakest in → track solved problems and revisit ones that took too long → redo a problem cold 2 weeks later to check real retention
2	**STAR-Format Answer Bank** — prepare behavioral answers before you're asked **Steps:** write 5 STAR-format answers using real ShopFlow decisions → cover: a disagreement, a mistake, a deadline, a hard bug, a proud moment → practice saying each out loud, not just reading it → time each to roughly 90 seconds
3	**Record a Project Walkthrough** — rehearse the most common opening question **Steps:** screen-record a 3-minute walkthrough of ShopFlow's architecture → watch it back and cut anything rambling → redo it once more, aiming for confident and concise → save it — this is also useful to link in applications
4	**Mock Technical Interview** — simulate real pressure **Steps:** find a peer, mentor, or use an AI/mock-interview tool → solve one DSA problem out loud, explaining your thinking as you go → solve one system-design-lite question about ShopFlow → get feedback on communication, not just correctness
5	**Job Posting Gap Analysis** — read postings critically instead of hopefully **Steps:** collect 5 real internship/junior postings you'd apply to → highlight every required skill you already have evidence for in ShopFlow → highlight any gap honestly → decide if the gap needs a quick project or is fine to learn on the job

■ Study Resources

- NeetCode: Interview Roadmap & Patterns — https://neetcode.io/roadmap
- Google: Interview Warmup (free, official) — https://grow.google/certificates/interview-warmup/
- Pramp: Free Peer Mock Interviews — https://www.pramp.com/

You're internship/job ready when: you can explain every phase of ShopFlow's architecture out loud, defend the technical decisions behind it, point to a live deployed URL, and solve a medium LeetCode problem under time pressure. That combination — not any single phase alone — is what gets you hired.

Full Progress Tracker

Every topic in this roadmap, in order — check each off as you complete its 5 projects

PART 0 · Foundations

- [ ] Java Anatomy & Syntax
- [ ] Terminal & Command Line Basics
- [ ] Git & GitHub Essentials
- [ ] Git Commands Reference
- [ ] UML Basics
- [ ] Dev Tools & Apps Roundup

PART A · Phase 1 · OOP Fundamentals

- [ ] Encapsulation
- [ ] Inheritance
- [ ] Abstraction
- [ ] Polymorphism
- [ ] Composition
- [ ] Exceptions
- [ ] SOLID Principles & Design Patterns

PART A · Phase 2 · Core Java Toolkit

- [ ] 2D & 3D Arrays
- [ ] Java Collections Framework
- [ ] Generics
- [ ] Exception Handling — Deep Dive
- [ ] File I/O
- [ ] Java Streams
- [ ] JDBC
- [ ] Build Tools
- [ ] Testing (JUnit & Mockito)
- [ ] Concurrency & Multithreading
- [ ] Logging

PART A · Phase 3 · Framework & API

- [ ] Spring Core & Dependency Injection
- [ ] REST API Design
- [ ] Request Validation & Exception Handling
- [ ] JWT Authentication
- [ ] GraphQL
- [ ] Testing Spring Boot Applications

PART A · Phase 4 · Databases

- [ ] JPA Entity Mapping & Relationships
- [ ] Lazy vs Eager Loading & N+1
- [ ] JPQL & Native Queries
- [ ] PostgreSQL Indexing & Optimization
- [ ] Transactions
- [ ] MongoDB & Document Databases
- [ ] Schema Migrations (Flyway)

PART B · Phase 5 · Frontend Foundations

- [ ] HTML
- [ ] CSS
- [ ] Bootstrap / Tailwind CSS
- [ ] JavaScript (ES6+)
- [ ] TypeScript

PART B · Phase 6 · Frontend Application Layer

- [ ] React + TypeScript
- [ ] Axios & API Integration
- [ ] Types of APIs
- [ ] Frontend Dev Tools

PART C · Phase 7 · External APIs

- [ ] Payment Integration
- [ ] Email Integration
- [ ] OAuth & Social Login
- [ ] File Storage & Uploads
- [ ] AI API Integration

PART C · Phase 8 · Deployment & Cloud

- [ ] Docker & Containerization
- [ ] CI/CD with GitHub Actions
- [ ] Cloud Compute Basics
- [ ] Cloud Storage
- [ ] PaaS Hosting
- [ ] Domains & HTTPS
- [ ] Monitoring & Error Tracking

PART D · Phase 9 · Data Structures & Algorithms

- [ ] Arrays & Strings
- [ ] Linked Lists
- [ ] Stacks & Queues
- [ ] Trees
- [ ] Heaps & Priority Queues
- [ ] Hashing
- [ ] Graphs
- [ ] Sorting & Searching
- [ ] Dynamic Programming

PART E · Phase 10 · Career & Interview Readiness

- [ ] System Design Basics
- [ ] Security Basics
- [ ] Resume & Portfolio
- [ ] Interview Preparation

That's the whole roadmap: 1 foundations part, 5 numbered parts, 10 phases, and roughly 45 topics — each with its own subtopics, 5 hands-on projects with step-by-step instructions, and 3+ real documentation sources. Two to three topics a week gets you through the entire thing in about a year, with one continuously growing, genuinely deployed project — ShopFlow — to show for it.